Git & GitHub — From Zero to Professional

A Complete Hands-On Guide for Absolute Beginners

and Working Professionals

Every expert was once a beginner. Every professional was once an amateur. Every master was once a disaster.

Dr M Quamrul Hassan MBBS, FCPS (Paediatrics)

Senior Consultant — Paediatrics & Neonatology Evercare Hospital Dhaka, Bangladesh

---

First Edition August 2026

About This Book

Who it is for: Anyone who wants to use Git and GitHub professionally — regardless of technical background. Written by a medical professional who learned Git from scratch, for everyone who thought Git was only for software developers.

What you will learn:

- How Git works and why it matters
- Every essential Git command — explained and practised
- How to collaborate using GitHub and pull requests
- How to automate workflows with GitHub Actions
- How to build and deploy a live website for free
- Professional practices used by real development teams

What you need:

- A computer (Windows, Mac or Linux)
- An internet connection
- No prior programming knowledge required

Book Structure

Section	Content
Foreword	The author's personal learning journey
Chapters 1-4	Git fundamentals — beginner level
Chapters 5-7	Branching, GitHub and collaboration
Chapters 8-9	Advanced Git and automation
Chapters 10-11	Real project and best practices
Annex A	Terminal and Markdown reference
Annex B	Complete Git cheat sheet

Annex C	Troubleshooting guide — 25 common problems
Annex D	Complete glossary

How to Read This Book

Complete beginner: Start at Chapter 1. Read every chapter in order. Open a terminal alongside the book and type every command.

Some experience: Read Chapters 1-2 quickly for vocabulary. Focus on Chapters 3-7 for core skills. Use Chapters 8-11 as needed.

Reference use: Jump directly to Annex B (cheat sheet) or Annex C (troubleshooting) for quick answers.

Study Plans

Two-week plan (recommended for beginners)

Week	Chapters	Focus
Week 1, Days 1-2	1, 2	Why Git exists and the vocabulary
Week 1, Days 3-4	3, 4	Installation and first repository
Week 1, Days 5-7	5	Branching — practice daily
Week 2, Days 1-2	6, 7	GitHub and collaboration
Week 2, Days 3-4	8	Advanced rescue commands
Week 2, Days 5-6	9, 10	Automation and real project
Week 2, Day 7	11	Best practices and next steps

One-week plan (intermediate)

Day	Chapters
Day 1	1, 2, 3
Day 2	4, 5
Day 3	6, 7
Day 4	8
Day 5	9, 10
Day 6	11, Annexes
Day 7	Review and practice

Weekend plan (experienced users)

Session	Chapters
Saturday morning	8 — Advanced rescue commands
Saturday afternoon	9 — GitHub Actions
Sunday morning	10, 11 — Project and best practices
Sunday afternoon	Annexes A, B, C, D

Connect with the Author

Website: mqhassan.github.io

GitHub: github.com/MQHassan

Research portfolio: github.com/MQHassan/research-portfolio

Professional CV: github.com/MQHassan/cv-biodata

Dedication

To every person who thought technology was not for them.

It is.

Acknowledgements

This book was written using the very tools it teaches — Git for version control, GitHub for hosting, Markdown for formatting and VS Code for editing.

Every chapter was committed, pushed and reviewed using Git. The book's repository is publicly available at:

https://github.com/MQHassan/git-github-book

You can see the complete writing history — every commit, every edit, every revision — as a living example of Git in professional use.

Licence

"The best time to learn Git was when you first started coding. The second best time is now."

Foreword

Why a Paediatrician Wrote a Book About Git

I am a doctor. A consultant paediatrician and neonatologist with over three decades of experience caring for sick children and newborns in Bangladesh. I have published 25 research papers. I have trained junior doctors, developed national curricula and led a paediatric intensive care unit.

I had never heard of Git.

Then one day, in a single intensive session, I decided to learn it. Not because someone told me to. Not because my job required it. But because I recognised that the world was changing and I wanted to understand the tools that were changing it.

What happened next surprised me.

The Journey

I started with zero knowledge. The first command I typed returned:

'git' is not recognized as an internal or external command, operable program or batch file.

By the end of that session I had:

- Installed Git on my Windows machine
- Created a repository and made my first commit
- Created branches, merged them and resolved my first conflict
- Pushed my work to GitHub
- Opened, reviewed and merged my first pull request
- Built a live professional website at mqhassan.github.io
- Set up an automated deployment pipeline using GitHub Actions

I had gone from that error message to a fully functioning professional presence on the internet — in one day.

Why This Book Is Different

Every Git book I encountered was written by a software developer for software developers. They assumed you already knew what a terminal was. They assumed you spoke the language of commits and branches as if these were obvious concepts.

They were not obvious to me.

As a doctor I had spent decades learning complex things — Anatomy, physiology, pathology, pharmacology, medicine, surgery, gynae, neonatal resuscitation, research methodology. I knew how to learn. What I needed was someone to explain Git the way a good teacher explains a difficult concept — with patience, with analogies, with examples drawn from real experience.

No one was writing that book.

So I wrote it.

What Makes This Book Work

This book was written from the inside of the learning experience.

Every analogy in these pages — the shopping basket for the staging area, the time machine for version control, the parallel universe machine for branching — emerged from my own struggle to understand these concepts. They are not developer metaphors. They are the explanations that actually worked for a non-developer trying to make sense of an unfamiliar world.

Every exercise in this book is real. I typed every command myself. I made every mistake. I hit every error. The troubleshooting guide in Annex C exists because I encountered those problems personally and had to find my way through them.

When I write "do not panic when you see a merge conflict" — I remember exactly how I felt the first time I saw those `<<<<<<<` markers appear in a file I thought I had broken.

Who This Book Is For

This book is for everyone who is not a software developer but wants to use the most powerful version control system in the world.

It is for the researcher who wants to track versions of their data analysis scripts.

It is for the medical professional who wants to maintain teaching materials that evolve over time.

It is for the writer who wants to version control a book manuscript — as I am doing with this very book.

It is for the business professional who wants to manage important documents with a proper audit trail.

It is for the student who wants to build a portfolio that employers can see.

It is for anyone who has ever saved a file as `final_ACTUAL_FINAL_v3.docx` and thought there must be a better way.

There is. You are holding it.

How to Use This Book

Read it with a terminal open.

Every concept is followed by real commands you can type immediately. The understanding comes from doing, not from reading. I learned Git by typing commands and seeing what happened. You will too.

Do not worry about making mistakes. In Git, almost nothing is truly lost. The troubleshooting guide in Annex C will get you out of any situation you get into.

Work through the chapters in order the first time. Each one builds on the previous. After that, use the book as a reference — the cheat sheet in Annex B, the troubleshooting guide in Annex C and the glossary in Annex D are designed to be returned to again and again.

A Note on Technology

All examples in this book use Windows Command Prompt and PowerShell, reflecting my own setup. Mac and Linux users will find the Git commands identical — only a handful of file management commands differ, and these are

noted throughout.

The Git version used throughout is 2.53.0. GitHub's interface changes occasionally — screenshots may differ slightly from what you see, but the concepts and workflows remain the same.

My Hope for You

When I started this journey I thought Git was something only programmers needed. I was wrong.

Git is a tool for anyone who creates things that change over time. That includes researchers, writers, educators, medical professionals, business analysts and countless others who have never written a line of code.

My hope is that by the end of this book you feel what I felt at the end of that first day — not just competent, but genuinely excited about what you can now build and share with the world.

Keep committing. Keep pushing. Keep learning.

Dr M Quamrul Hassan MBBS, FCPS (Paediatrics) Senior Consultant — Paediatrics & Neonatology Evercare Hospital Dhaka, Bangladesh

mqhassan.github.io

August 2026

Chapter 1 — Why Git Exists

Learning Objectives

By the end of this chapter you will be able to:

- Explain the problem that Git was created to solve
- Describe what version control means in plain English
- Understand the difference between Git and GitHub
- Identify real-world situations where Git is essential
- Explain why Git became the world standard for version control

---

1.1 The Problem — Life Before Git

Imagine you are writing an important research paper. You save it as:

research_final.docx

Then you make some changes. Not sure if they are good, so you save a copy:

research_final_v2.docx

Your supervisor suggests edits. You save another copy:

research_final_v2_supervisor_edits.docx

You make more changes. You save:

research_final_ACTUAL_FINAL.docx

Then you realise you preferred version 2. But which one was version 2? And what exactly did you change between versions?

Now imagine this same chaos — but with 10 people all editing the same document simultaneously. Who changed what? When? Why? Which version is the real one?

This is the problem Git solves.

1.2 What is Version Control?

Version control is a system that:

- Records every change ever made to a file
- Remembers who made each change
- Remembers when each change was made
- Remembers why each change was made (through messages)
- Allows you to go back to any previous version at any time
- Allows multiple people to work on the same files without conflict

Think of it like this:

> *A version control system is like a time machine combined with a detailed diary for your files.*

1.3 What is Git?

Git is the world's most popular version control system. It was created in 2005 by **Linus Torvalds** — the same person who created the Linux operating system — to manage the development of Linux, which involved thousands of developers around the world.

Key facts about Git

Fact	Detail
Created	2005
Creator	Linus Torvalds
Type	Distributed version control system
Cost	Free and open source
Used by	Over 100 million developers worldwide
Runs on	Windows, Mac, Linux

What makes Git special

Before Git, version control systems were **centralised** — one server held the master copy and everyone had to connect to it to work. If the server went down, work stopped.

Git changed everything by being **distributed** — every developer has a complete copy of the entire history on their own machine. You can work on a plane with no internet, make dozens of commits, and sync up later.

Centralised (old way): Distributed (Git's way):

[Server] [Your machine — full history] / | \ |

Dev1 Dev2 Dev3 [Team member — full history] | Everyone depends [Team member — full history] on the server

---

1.4 What is GitHub?

Git and GitHub are **not the same thing**. This confuses almost every beginner. Here is the clear distinction:

	Git	GitHub
What it is	Software on your computer	A website on the internet
What it does	Tracks changes to your files	Hosts your Git repositories online
Who made it	Linus Torvalds (2005)	GitHub Inc, now owned by Microsoft (2008)
Cost	Always free	Free for public repos, paid plans for private
Can you use one without the other?	Yes — Git works without GitHub	No — GitHub needs Git

A simple analogy:

> *Git is like Microsoft Word — software that runs on your computer. GitHub is like Google Drive — a place to store and share your files online. You use Word to create and edit documents. You use Google Drive to back them up and share them with others.*

1.5 Real World Use Cases

Git is not just for software developers. Here are real situations where Git makes a significant difference:

For researchers and academics

- Version control your research papers through multiple drafts
- Track every change to your data analysis scripts
- Collaborate with co-authors without emailing files back and forth
- Maintain a public portfolio of your research on GitHub

For medical professionals

- Manage clinical guidelines that get updated regularly
- Track changes to protocols and standard operating procedures
- Maintain teaching materials that evolve over time
- Build a professional online presence for your practice

For software developers

- Track every line of code ever written
- Work on new features without breaking the main product
- Review and approve changes before they go live
- Automate testing and deployment

For writers and content creators

- Version control books, articles and scripts
- Experiment with different structures without losing the original
- Collaborate with editors and co-authors
- Publish content directly from a repository

For business professionals

- Track changes to important documents and reports
- Maintain organised project histories
- Collaborate across teams and time zones

1.6 Why Git Won

There were other version control systems before Git — CVS, SVN, Perforce, Mercurial. Git beat them all. Here is why:

Reason	Explanation
Speed	Git is incredibly fast — even on huge projects
Distributed	Every copy is complete — no single point of failure
Branching	Creating parallel versions is instant and cheap

Free	No cost, no licence, no restrictions
Community	GitHub built a social layer that made Git the standard
Industry adoption	Every major tech company uses Git

Today Git is not optional for anyone working in technology. It is the air the industry breathes.

1.7 The Mental Model to Remember

Before moving to the next chapter, fix this picture in your mind: Git = time machine + parallel universe machine for your files

Time machine: Go back to any point in your project's history See exactly what changed between any two points Recover anything that was ever saved

Parallel universe machine (branching): Create an alternate version of your project Experiment freely without affecting the original Merge the best version back when ready

GitHub is where these timelines meet and are shared with the world.

Chapter Summary

Concept	Key point
Version control	Records every change, who made it, when and why
Git	Free distributed version control system created in 2005
GitHub	Website that hosts Git repositories online
Git vs GitHub	Git is the tool, GitHub is the platform
Why Git	Speed, distribution, branching, free, universal adoption

---

Assessment — Test Yourself

Answer these questions before checking the answers below.

Question 1 What problem does Git solve that saving files with different names (v1, v2, final, ACTUAL_FINAL) does not?

Question 2 What is the difference between a centralised and a distributed version control system?

Question 3 Your colleague says "I use GitHub to track my code changes." What is technically incorrect about this statement?

Question 4 Name three professions outside software development that can benefit from using Git.

Question 5 Linus Torvalds created Git in 2005. What was his original reason for creating it?

Answers

Answer 1 Git records the complete history of every change with the exact lines that changed, who changed them, when, and a written message explaining why. With named files you lose all this context — you cannot see what changed between versions, who made changes, or why decisions were made. Git also allows multiple people to work on the same files simultaneously without overwriting each other's work.

Answer 2 A centralised system has one server that holds the master copy — everyone must connect to it to work. If the server goes down, work stops. A distributed system (like Git) gives every user a complete copy of the entire history on their own machine. Work continues even without internet access. There is no single point of failure.

Answer 3 GitHub does not track code changes — Git does. GitHub is a website that hosts Git repositories online. The correct statement would be: "I use Git to track my code changes and GitHub to host and share them."

Answer 4 Any three from: researchers, academics, medical professionals, writers, content creators, business professionals, lawyers, educators, journalists, data scientists, designers.

Answer 5 Linus Torvalds created Git to manage the development of the Linux kernel, which involved thousands of developers around the world contributing code. The existing version control systems were too slow, centralised and limited for a project of that scale.

What is Next

In Chapter 2 we learn the language of Git — every important term explained in plain English with analogies. Understanding the vocabulary makes everything that follows much easier.

Proceed to Chapter 2 — The Language of Git

Chapter 2 — The Language of Git

Learning Objectives

By the end of this chapter you will be able to:

- Define every core Git term in plain English
- Explain each term using a real-world analogy
- Distinguish between commonly confused terms
- Use Git vocabulary confidently in conversation
- Pass the chapter assessment with full marks

2.1 Why Vocabulary Matters

Every field has its own language. Medicine has terms like tachycardia, myocardial infarction and haemodynamic compromise. Git has terms like repository, commit, branch and HEAD.

Just as medical terminology helps doctors communicate precisely and efficiently, Git terminology helps developers describe complex operations in a single word.

The good news — Git has far fewer terms than medicine. Master these 25 terms and you will understand everything you read and hear about Git.

2.2 The Core Terms

Repository (repo)

What it is: A folder that Git is tracking. Contains all your project files plus a hidden `.git` folder that stores the entire history of every change ever made.

Analogy:

> *A repository is like a patient's complete medical record — it contains not just the current state of the patient, but every consultation, test result and treatment ever recorded.*

In practice:

my-project/ ← this is your repository index.html style.css .git/ ← hidden folder — Git's brain

Working Tree

What it is: The actual files on your disk that you are currently editing. What you see in your file explorer and text editor.

Analogy:

> *The working tree is your physical desk — where active work happens. Papers on the desk are being worked on right now.*

Key point: Changes in the working tree are not saved to Git history until you explicitly tell Git to record them.

Staging Area (Index)

What it is: A preparation zone where you place changes before committing them. Lets you choose exactly which changes go into a commit — you do not have to commit everything at once.

Analogy:

> *The staging area is like a shopping basket in a supermarket. You add items to the basket before going to the checkout. You can put things in, take things out, and only pay (commit) when you are happy with what is in the basket.*

The three zones of Git:

Working Tree Staging Area Repository (your files) --> (the basket) --> (permanent history)

git add git commit

Commit

What it is: A saved snapshot of your project at a specific moment. Each commit has a unique ID, a message, an author, and a timestamp. Commits are permanent — they cannot be accidentally overwritten.

Analogy:

> *A commit is like a save point in a video game. You can always return to any save point, no matter how many moves you make after it.*

What a commit contains:

- A unique ID (SHA hash) — e.g. `a3f9c2b`
- Your name and email
- The date and time
- A message describing what changed
- A snapshot of all the changes

SHA Hash

What it is: The unique 40-character ID that identifies each commit. Usually shortened to the first 7 characters. Example: `a3f9c2b`

Analogy:

> *A SHA hash is like a patient's hospital ID number — completely unique, never repeated, identifies one specific record.*

Why it matters: You use the SHA to refer to specific commits when you want to go back in time, compare versions or cherry-pick changes.

HEAD

What it is: A special pointer that tells Git "this is where you are right now." HEAD usually points to the tip of your current branch.

Analogy:

> *HEAD is the "You Are Here" marker on a map. As you move through your project history, HEAD moves with you.*

Important: When you make a new commit, HEAD automatically moves forward to point at the new commit.

Commit 1 --> Commit 2 --> Commit 3 ^ HEAD (you are here)

Branch

What it is: An independent line of development. A lightweight pointer to a specific commit. You can have many branches at once. Creating a branch is instant and takes almost no disk space.

Analogy:

> *A branch is like a parallel timeline in your project's history. The main timeline continues undisturbed while you work in an alternate timeline. When you are happy with the result, you merge the timelines back together.*

Key insight: A branch is just a label — a pointer to a commit. It is not a copy of your files. This is why Git branches are so fast and cheap compared to older systems.

---

main / master

What it is: The default branch of a repository. `master` was the old default name. `main` is now the standard on GitHub.

Analogy:

> *The main branch is like the trunk of a tree — the official, stable version. All other branches grow from it and eventually merge back into it.*

Best practice: Never commit directly to main in a team setting. Always use a separate branch and merge via a pull request.

Merge

What it is: Combining the changes from one branch into another. Creates a merge commit if there are diverging histories.

Analogy:

> *Merging is like two rivers joining into one. The water from both rivers flows together from that point.*

Types of merge:

Type	When it happens	What it creates
Fast-forward	When the base branch has no new commits	No new commit — just moves the pointer
3-way merge	When both branches have new commits	A new merge commit

---

Merge Conflict

What it is: When Git cannot automatically merge two changes because they touch the same lines of the same file. You must resolve it manually.

Analogy:

> *A merge conflict is like two doctors writing different diagnoses in the same field of a patient's notes. A human must decide which diagnosis is correct.*

What a conflict looks like in a file:

<<<<<<< HEAD Git is a version control system

Git is a powerful version control tool

feature-branch You must edit the file to keep the correct version, remove the conflict markers, then commit the resolution.

Rebase

What it is: Re-applies your commits on top of another branch, creating a cleaner linear history. Rewrites commit history.

Analogy:

> *Rebase is like picking up your work and moving it to a new starting point — as if you had started working from the latest version all along.*

Caution: Never rebase commits that have already been pushed to a shared branch. It rewrites history and causes problems for teammates.

Stash

What it is: Temporarily shelves changes you are not ready to commit, so you can switch branches with a clean working tree.

Analogy:

> *Stash is like a drawer you shove work into when guests arrive unexpectedly. The work is safe, out of the way, and you can retrieve it exactly as you left it when you are ready.*

Remote

What it is: A version of your repository hosted elsewhere — typically on GitHub. You push to and pull from remotes.

Analogy:

> *A remote is like a shared copy of your project living in the cloud. Your local repo is your private copy. The remote is the shared copy everyone can access.*

Origin

What it is: The conventional name for your main remote — the one you cloned from or connected to. You can have multiple remotes with different names but origin is always the primary one.

Analogy:

> *Origin is like saving a phone number under the name "Home" — it is just a convenient shortcut name for a long address.*

The full address:

origin = https://github.com/MQHassan/my-project.git

Instead of typing that URL every time you just type `origin`.

Clone

What it is: Downloads a full copy of a remote repository to your machine, including all history and all branches.

Analogy:

> *Cloning a repo is like photocopying an entire filing cabinet — you get every document, every note, every record — a complete independent copy.*

Push

What it is: Sends your local commits up to the remote repository. Shares your work with the team or backs it up online.

Analogy:

> *Push is like uploading your work to the shared server. Once pushed, everyone with access can see your commits.*

Pull

What it is: Fetches changes from the remote and immediately merges them into your current branch. Shorthand for fetch + merge.

Analogy:

> *Pull is like downloading and installing the latest update — it gets the new content AND applies it in one step.*

Fetch

What it is: Downloads changes from the remote but does NOT merge them. Lets you review what changed before integrating.

Analogy:

> *Fetch is like downloading your emails without opening them. They arrive on your machine but you have not read or acted on them yet.*

Fetch vs Pull:

Command	Downloads	Merges	Safe?
git fetch	Yes	No	Always safe
git pull	Yes	Yes	Usually safe

Fork

What it is: A personal copy of someone else's GitHub repository. Changes in your fork do not affect the original until you submit a pull request.

Analogy:

> *Forking is like photocopying a published book so you can write in the margins. Your copy is independent. If your annotations are valuable, you can suggest they go into the next edition (pull request).*

Fork vs Clone:

	Fork	Clone
Where	GitHub (server side)	Your computer (local)
Purpose	Contribute to others' projects	Work on any repo locally
Affects original?	No	No

Pull Request (PR)

What it is: A GitHub feature to propose merging a branch into another. The hub of code review — comments, approvals and checks happen here before any merge.

Analogy:

> *A pull request is like raising your hand and saying: "I have made some changes — please review them before we add them to the official record."*

.gitignore

What it is: A file listing paths and patterns that Git should never track. Sensitive files, dependencies and build outputs go here.

Analogy:

> *.gitignore is a "do not photograph" list for Git. Anything on the list is completely invisible to Git.*

Common entries:

node_modules/ ← dependencies — huge and regeneratable .env ← environment secrets — never commit these! *.log ← log files dist/ ← build output .DS_Store ← Mac system files

---

Tag

What it is: A permanent, named pointer to a specific commit. Used to mark release versions like v1.0.0.

Analogy:

> *A tag is like a sticky label on a specific point in history. Unlike branches which move forward, tags stay fixed forever.*

Reflog

What it is: A local log of every time HEAD moved — every commit, reset, merge and switch. Lets you recover commits even after accidental deletion.

Analogy:

> *Reflog is Git's black box recorder — it captures everything, even things that appear to have been deleted.*

2.3 Commonly Confused Pairs

These pairs trip up almost every beginner. Study them carefully.

Git vs GitHub

Git = the tool (runs on your computer) GitHub = the platform (website that hosts repos)

Commit vs Save

Save = updates the file on disk (Ctrl+S) Commit = records a permanent snapshot in Git history

Fetch vs Pull

Fetch = downloads changes, does NOT apply them Pull = downloads AND applies changes (fetch + merge)

Fork vs Clone

Fork = server-side copy on GitHub Clone = local copy on your machine

Merge vs Rebase

Merge = combines histories, preserves branching record Rebase = rewrites history to appear linear

Branch vs Tag

Branch = a moving pointer — advances with new commits Tag = a fixed pointer — stays on one commit forever

---

2.4 The Complete Git Vocabulary Map

Your machine GitHub (the internet)

Working tree Remote repository | | | git add | git push (upload) v | git pull (download) Staging area | | Origin (the remote name) | git commit | v Forked repos Local repository Pull requests | HEAD (where you are) | Branches (parallel timelines) | Commits (permanent snapshots) | SHA hashes (unique IDs)

Chapter Summary

Term	One-line definition
Repository	A folder Git is tracking with full history
Working tree	Your files on disk being actively edited
Staging area	Preparation zone before committing
Commit	A permanent snapshot with ID, message and timestamp
SHA hash	The unique ID of every commit
HEAD	The pointer showing where you are right now
Branch	A lightweight pointer to a commit — a parallel timeline
main	The default primary branch

Merge	Combining two branches into one
Merge conflict	When Git cannot auto-merge — human must decide
Stash	Temporary shelf for unfinished work
Remote	A hosted copy of your repo (e.g. on GitHub)
Origin	The conventional name for your primary remote
Clone	Download a full copy of a remote repo
Push	Upload your commits to the remote
Pull	Download and apply remote changes
Fetch	Download remote changes without applying
Fork	A personal server-side copy of someone's repo
Pull request	A proposal to merge a branch — with review
.gitignore	A list of files Git should never track
Tag	A fixed permanent label on a specific commit
Reflog	Git's complete movement history — the black box

Assessment — Test Yourself

Question 1 What is the difference between the working tree and the staging area?

Question 2 You run `git fetch`. Your colleague runs `git pull`. What is the practical difference in what happened on each machine?

Question 3 Your project has a file called `.env` containing your database password. How do you ensure Git never tracks this file?

Question 4 Explain what HEAD is using your own analogy — different from the one in this chapter.

Question 5 You want to contribute to an open source project on GitHub that you do not own. What is the correct first step — fork or clone? Explain why.

Question 6 What does the SHA hash `a3f9c2b` represent and why is it useful?

Question 7 What is the difference between a branch and a tag?

Answers

Answer 1 The working tree contains your files as they currently exist on disk — actively being edited. The staging area is a preparation zone where you explicitly place changes you want to include in the next commit. Changes in the working tree are invisible to Git history until you run `git add` to move them to the staging area.

Answer 2 Your colleague's machine downloaded AND applied (merged) the remote changes into their current branch. Your machine only downloaded the changes — your working files are unchanged. You can review what

arrived before deciding to merge. Running `git pull` is equivalent to running `git fetch` followed by `git merge`.

Answer 3 Add `.env` to a file called `.gitignore` in the root of your repository. Once listed there, Git will completely ignore the file — it will never appear in `git status`, never be staged and never be committed. The `.gitignore` file itself should be committed so the rule applies to everyone working on the project.

Answer 4 Accept any reasonable analogy. Example: HEAD is like a bookmark in a book — it marks exactly where you are currently reading. As you turn pages (make commits) the bookmark moves with you. You can move the bookmark to any page (any commit) at any time.

Answer 5 Fork first. Forking creates your own server-side copy of the project on GitHub under your account. You then clone YOUR fork to your local machine. This way you can push changes to your fork without needing permission from the original project owner. When your changes are ready, you open a pull request from your fork to the original repository.

Answer 6 `a3f9c2b` is the short form of the SHA hash — the unique identifier of a specific commit. It is useful because it lets you refer to any specific point in history precisely. You use it to check out old versions, compare changes, revert to a previous state or cherry-pick a specific commit.

Answer 7 A branch is a moving pointer — as you make new commits on a branch, the pointer advances to the latest commit. A tag is a fixed pointer — it permanently marks one specific commit and never moves. Branches are for ongoing work. Tags are for marking milestones like software releases (v1.0, v2.0).

What is Next

In Chapter 3 we stop talking and start doing. You will install Git on your machine, configure it with your identity, and verify everything is working correctly.

Proceed to Chapter 3 — Installation and Setup

Chapter 3 — Installation and Setup

Learning Objectives

By the end of this chapter you will be able to:

- Install Git on Windows, Mac or Linux
- Verify the installation was successful
- Configure Git with your identity
- Set your preferred text editor
- Understand what each configuration setting does
- Fix the most common installation problems

3.1 Before You Install

Git is free software. You do not need to pay for it, register for anything or provide personal information to download it.

The official source for Git is:

https://git-scm.com

Always download from this official source. Never from third-party websites.

What you will install

Component	What it is
Git	The core version control software
Git Bash (Windows only)	A terminal that lets you use Unix commands on Windows
Git GUI (optional)	A basic graphical interface for Git

3.2 Installing Git on Windows

Step 1 — Download

Go to:

https://git-scm.com/download/win

The download starts automatically. Look for a file named:

Git-2.x.x-64-bit.exe

Download the 64-bit version for modern Windows machines.

Step 2 — Run the installer

Double-click the downloaded `.exe` file. Windows may ask: "Do you want to allow this app to make changes?" Click **Yes**.

Step 3 — Key installer choices

You will see approximately 10 screens. Most can be left at default. These three screens matter:

Screen: Choose default editor

- Change from Vim to **Visual Studio Code**
- Vim is extremely difficult for beginners to exit

Screen: Override the default branch name

- Select **Override to main**
- This matches GitHub's default and is the modern standard

Screen: Adjusting your PATH environment

- Select **Git from the command line and also from 3rd-party software**
- This lets you use `git` in Command Prompt, not just Git Bash

All other screens — leave at recommended defaults and click Next.

Step 4 — Complete the installation

Click **Install**. The process takes about 30 to 60 seconds.

On the final screen:

- Uncheck **View Release Notes**
- Click **Finish**

Step 5 — Verify the installation

Critical: Close your current Command Prompt completely and open a brand new one. Windows only recognises new programs in new windows.

In the new Command Prompt type:

git --version

You should see something like:

git version 2.45.0.windows.1

If you see this — Git is installed correctly.

Common Windows installation problems

Problem	Cause	Fix
`git` is not recognised	Old Command Prompt still open	Close all windows, open a new one
`git` still not recognised	PATH not set correctly	Restart your computer
Download fails	Network issue	Try a different browser or network

---

3.3 Installing Git on Mac

Mac users have two options:

Option 1 — Xcode Command Line Tools (recommended for beginners)

Open Terminal (search for Terminal in Spotlight) and type:

git --version

If Git is not installed, Mac will automatically offer to install the Xcode Command Line Tools. Click **Install** and wait for it to complete. This installs Git automatically.

Option 2 — Homebrew (recommended for power users)

If you have Homebrew installed:

brew install git

Verify on Mac

git --version

You should see:

git version 2.x.x (Apple Git-xxx)

3.4 Installing Git on Linux

Ubuntu / Debian

sudo apt update sudo apt install git

Fedora / Red Hat

sudo dnf install git

Arch Linux

sudo pacman -S git

Verify on Linux

git --version

3.5 First-Time Configuration

After installation, Git needs to know who you are. Every commit you make will be permanently stamped with this information.

You only need to do this once per machine.

Set your name

git config --global user.name "Your Full Name"

Set your email

git config --global user.email "you@example.com"

Set the default branch name

git config --global init.defaultBranch main

Set your default editor

For Visual Studio Code:

git config --global core.editor "code --wait"

For Notepad++ (Windows):

git config --global core.editor "'C:/Program Files/Notepad++/notepad++.exe' -multiInst -notabbar -nosession -noPlugin"

For Nano (Linux/Mac):

git config --global core.editor nano

What does --global mean?

The `--global` flag means this setting applies to ALL repositories on your machine. Without it the setting only applies to the current repository.

Flag	Scope
`--global`	All repos on your machine
`--local` (default)	Current repo only
`--system`	All users on the machine

3.6 Verifying Your Configuration

After setup, confirm everything saved correctly:

git config --list

You should see output similar to:

user.name=Dr M Quamrul Hassan user.email=mqh.pt@google.com init.defaultbranch=main core.editor=code --wait

To check a single setting:

git config user.name

3.7 Understanding the Configuration File

Your global Git configuration is stored in a file called `.gitconfig` in your home directory.

On Windows:

C:\Users\YourName.gitconfig

On Mac/Linux:

~/.gitconfig

You can open and edit this file directly:

git config --global --edit

A typical `.gitconfig` file looks like:

[user] name = Dr M Quamrul Hassan email = mqh.pt@google.com [init] defaultBranch = main [core] editor = code --wait

---

3.8 Setting Up Visual Studio Code

Visual Studio Code (VS Code) is the recommended editor for working with Git. It is free, powerful and integrates deeply with Git.

Download VS Code

https://code.visualstudio.com

Why VS Code for Git?

Feature	Benefit
Built-in Git panel	Visual view of changes, staging and commits
Conflict resolution	Visual merge conflict editor
Markdown preview	See formatted output as you write
Integrated terminal	Run Git commands without leaving the editor
UTF-8 default	Files saved with correct encoding for GitHub
Extensions	Thousands of add-ons for every language

The `code` command

After installing VS Code, you can open it from the terminal:

code . ← open current folder in VS Code code README.md ← open a specific file code myproject/ ← open a specific folder

If `code` is not recognised in your terminal:

1. Open VS Code
2. Press `Ctrl+Shift+P`
3. Type: `Shell Command: Install 'code' command in PATH`
4. Press Enter
5. Close and reopen your terminal

3.9 Useful Git Aliases

Aliases are shortcuts for long commands. Set them once and use them forever.

git config --global alias.st status git config --global alias.co checkout git config --global alias.br branch git config --global alias.lg "log --oneline --graph --all"

After setting these:

Instead of	You can type
`git status`	`git st`
`git checkout`	`git co`
`git branch`	`git br`
`git log --oneline --graph --all`	`git lg`

The `git lg` alias is particularly useful — it shows a beautiful visual graph of your entire branch history.

3.10 Installation Checklist

Before proceeding to Chapter 4, confirm all of these:

[ ] git --version shows a version number [ ] git config user.name returns your name [ ] git config user.email returns your email [ ] git config init.defaultBranch returns main [ ] VS Code is installed [ ] code . opens VS Code from the terminal

If any item is unchecked, go back to the relevant section and complete it before proceeding.

Chapter Summary

Topic	Key point
Official source	Always download Git from git-scm.com
Windows installer	3 key choices: editor, branch name, PATH
Verification	Run `git --version` in a NEW terminal window
Configuration	Set name, email, branch and editor once with --global
VS Code	Recommended editor — UTF-8, integrated Git, terminal
Aliases	Shortcuts that save time on common commands

Assessment — Test Yourself

Question 1 After installing Git on Windows, you open your existing Command Prompt and type `git --version`. You see `git is not recognised`. What is the most likely cause and how do you fix it?

Question 2 What is the difference between `git config --global` and `git config --local`?

Question 3 Why is it important to use the same email address for Git configuration as the one you use for your GitHub account?

Question 4 You set up Git on your work laptop last year. Today you get a new personal laptop. Do you need to run `git config --global` again? Explain your answer.

Question 5 What does the command `git config --list` do and when would you use it?

Answers

Answer 1 The most likely cause is that the existing Command Prompt window was open before Git was installed. Windows only loads the updated PATH (which includes Git) when a new terminal window is opened. Fix: Close all Command Prompt windows completely and open a brand new one. If the problem persists, restart the computer.

Answer 2 `git config --global` sets a configuration that applies to ALL repositories on your machine. It is stored in a `.gitconfig` file in your home directory. `git config --local` (the default when no flag is specified) sets a configuration that only applies to the current repository. It is stored in `.git/config` inside the repo.

Answer 3 GitHub uses your email address to connect your commits to your GitHub profile. If the email in your Git config matches your GitHub account email, your commits appear on your GitHub profile with your avatar and username. If they do not match, commits appear as unlinked contributions from an unknown author.

Answer 4 Yes. The `--global` configuration is stored per machine, not per account. Your new laptop has no Git configuration. You must run the `git config --global` commands again on every new machine you use. However, your repositories on GitHub are unaffected — they contain the full history from all your previous machines.

Answer 5 `git config --list` displays all current Git configuration settings — name, email, editor, branch defaults, aliases and more. Use it to verify your setup after configuring Git for the first time, or to diagnose problems when Git behaves unexpectedly.

What is Next

In Chapter 4 we create our very first Git repository from scratch. Every concept from Chapters 1 and 2 becomes real as you type your first commands and make your first commit.

Proceed to Chapter 4 — Your First Repository

Chapter 4 — Your First Repository

Learning Objectives

By the end of this chapter you will be able to:

- Create a new Git repository from scratch
- Understand the three zones of Git in practice
- Stage and commit files with confidence
- Read and interpret git status output
- Read and interpret git diff output
- View your commit history with git log
- Undo changes safely before committing
- Create and use a .gitignore file

4.1 The Three Zones — A Quick Reminder

Before typing a single command, fix this picture in your mind. Every Git operation moves changes between these three zones:

Zone 1 Zone 2 Zone 3 Working Tree --> Staging Area --> Repository (your files) (the basket) (permanent history)

git add git commit

Working tree — files on your disk, actively being edited
Staging area — changes queued up for the next commit
Repository — permanent history, safe forever

Everything in this chapter is about moving changes between these three zones.

4.2 Creating Your First Repository

Open your terminal (Command Prompt on Windows, Terminal on Mac/Linux).

Step 1 — Navigate to your Desktop

Windows:

cd C:\Users\YourName\Desktop

Mac/Linux:

cd ~/Desktop

Step 2 — Create a project folder

mkdir my-first-repo cd my-first-repo

Step 3 — Initialise Git

git init

You will see:

Initialized empty Git repository in /Desktop/my-first-repo/.git/

What just happened: Git created a hidden `.git` folder inside `my-first-repo`. That folder IS Git's entire database for this project. It stores every commit, every branch, every piece of history.

> *Warning: Never delete the `.git` folder. Deleting it permanently destroys your entire project history. The files remain but Git forgets everything.*

Step 4 — Verify

git status

You will see:

On branch main No commits yet nothing to commit (create/copy files and use "git add" to track)

Read this output carefully:

- `On branch main` — you are on the main branch
- `No commits yet` — this repo has zero history
- `nothing to commit` — no files exist yet

Git is even helpful enough to tell you what to do next.

4.3 Your First File and Commit

Create a file

Windows:

echo Hello from my first Git repo > readme.txt

Mac/Linux:

echo "Hello from my first Git repo" > readme.txt

Check the status

git status

Output:

On branch main

No commits yet

Untracked files: (use "git add ..." to include in what will be committed) readme.txt

nothing added to commit but untracked files present

Reading this output:

Line	Meaning
`Untracked files`	Git sees the file but is not watching it
`readme.txt`	The specific file that is untracked

`use "git add"`	Git tells you exactly what to do next

The file is in Zone 1 (working tree) but Git is not tracking it.

Stage the file

git add readme.txt

Check status again

git status

Output:

On branch main

No commits yet

Changes to be committed: (use "git restore --staged ..." to unstage) new file: readme.txt

What changed:

- `Changes to be committed` — the file moved to Zone 2 (staging area)
- `new file: readme.txt` — Git will record this as a brand new file

Make your first commit

git commit -m "Add readme file"

Output:

[main (root-commit) a3f9c2b] Add readme file 1 file changed, 1 insertion(+) create mode 100644 readme.txt

Reading this output:

Part	Meaning
`main`	The branch this commit is on
`root-commit`	This is the very first commit — no parent
`a3f9c2b`	The short SHA hash — unique ID
`Add readme file`	Your commit message
`1 file changed`	One file was added
`1 insertion(+)`	One line was added

Congratulations — you have made your first Git commit.

4.4 Viewing History

git log

This shows the full commit history. Press `q` to exit.

For a cleaner view:

git log --oneline

Output:

a3f9c2b (HEAD -> main) Add readme file

Read this:

- `a3f9c2b` — short SHA hash
- `HEAD -> main` — you are here, on main branch
- `Add readme file` — your commit message

As you make more commits the log grows upward — newest at the top.

4.5 Making More Commits

Let us practice the full workflow again.

Add a second line to the file

Windows:

echo This is my Git practice project >> readme.txt

Mac/Linux:

echo "This is my Git practice project" >> readme.txt

Note: `>>` appends to the file. `>` would overwrite it.

See what changed

git diff

Output:

diff --git a/readme.txt b/readme.txt --- a/readme.txt +++ b/readme.txt @@ -1 +1,2 @@ Hello from my first Git repo +This is my Git practice project

Reading git diff:

Symbol	Meaning
---	The old version of the file
+++	The new version of the file
(space)	This line is unchanged
+	This line was added (shown in green)
-	This line was removed (shown in red)

Stage and commit

git add readme.txt git commit -m "Add project description to readme"

View the updated history

git log --oneline

Output:

b8d2f1a (HEAD -> main) Add project description to readme a3f9c2b Add readme file

Two commits. Your history is growing.

4.6 Staging Selected Changes

One of Git's most powerful features is that you do not have to commit everything at once. You can choose exactly which changes go into each commit.

Stage a specific file

git add readme.txt

Stage all changed files at once

git add .

The dot means "everything in the current folder and subfolders."

Stage parts of a file (patch mode)

git add -p readme.txt

This enters interactive mode where Git shows you each change individually and asks if you want to stage it. Type:

- `y` to stage this change
- `n` to skip this change
- `q` to quit

Why this matters: Imagine you made two unrelated changes in one file — fixed a typo AND added a new feature. Good commits are focused on one thing. Patch mode lets you put the typo fix in one commit and the feature in another, even though they are in the same file.

4.7 Reading git status Like a Pro

`git status` is the command you will run more than any other. Here is how to read every possible output:

Clean state

On branch main nothing to commit, working tree clean

Meaning: Everything is committed. No changes anywhere.

Untracked files

Untracked files: newfile.txt

Meaning: Git sees this file but has never tracked it. Action: `git add newfile.txt`

Modified file (not staged)

Changes not staged for commit: modified: readme.txt

Meaning: A tracked file has been changed but not staged. Action: `git add readme.txt`

Staged file

Changes to be committed: modified: readme.txt

Meaning: Change is in the staging area, ready to commit. Action: `git commit -m "your message"`

Mixed state (common in real work)

Changes to be committed: modified: readme.txt

Changes not staged for commit: modified: style.css

Untracked files: newpage.html

Meaning: readme.txt is staged, style.css is modified but not staged, newpage.html has never been tracked.

4.8 Undoing Changes

Git provides several ways to undo changes depending on where in the three zones you are.

Undo changes in the working tree (before staging)

Restore a file to its last committed state:

git restore readme.txt

Restore ALL files to their last committed state:

git restore .

> *Warning: This permanently discards uncommitted changes. There is no undo for this command. Use with care.*

Unstage a file (move back from staging to working tree)

git restore --staged readme.txt

This removes the file from the staging area but keeps your changes. The file goes back to Zone 1 (working tree).

Fix the last commit message (before pushing)

git commit --amend -m "Corrected commit message"

> *Warning: Only use this on commits you have NOT yet pushed. Amending a pushed commit causes problems for teammates.*

4.9 The .gitignore File

Some files should never be tracked by Git:

- Password and API key files (`.env`)
- Dependency folders (`node_modules/`)

- Build output (`dist/` , `build/`)
- Editor config files (`.vscode/` , `.idea/`)
- Operating system files (`.DS_Store` , `Thumbs.db`)

Creating a .gitignore file

Create a file called `.gitignore` in the root of your repo:

Windows:

echo .env > .gitignore

Or open it in your editor and add entries: Dependencies

node_modules/

Environment secrets

.env .env.local

Build output

dist/ build/

Editor files

.vscode/ .idea/

OS files

.DS_Store Thumbs.db

Lines starting with `#` are comments — they are ignored by Git.

Testing it works

Create a secret file:

echo my_password > secret.txt

Add `secret.txt` to `.gitignore` :

echo secret.txt >> .gitignore

Now run:

git status

`secret.txt` will not appear — Git is completely ignoring it.

Committing .gitignore

Always commit your `.gitignore` file:

git add .gitignore git commit -m "Add gitignore to exclude sensitive files"

This ensures everyone working on the project shares the same ignore rules.

What if I already committed a file I want to ignore?

Adding a file to `.gitignore` after it has been committed does not un-track it. You must explicitly tell Git to stop tracking it:

git rm --cached secret.txt git commit -m "Stop tracking secret.txt"

The file stays on your disk but Git no longer watches it.

4.10 The Complete Everyday Workflow

This is the rhythm you will repeat hundreds of times. Memorise it. It becomes muscle memory very quickly.

Step 1: Check where you are and what has changed git status

Step 2: Review the exact changes git diff

Step 3: Stage the changes you want in this commit git add filename.txt (or git add . for everything)

Step 4: Verify what is staged git diff --staged

Step 5: Commit with a clear message git commit -m "Describe what changed and why"

Step 6: Confirm it is in history git log --oneline

Writing good commit messages

A commit message should complete this sentence: **"If applied, this commit will..."**

Good message	Bad message
Add user login page	stuff
Fix navigation bar alignment bug	fixed it
Update readme with installation steps	update
Remove deprecated payment function	changes

Format:

- Start with a verb in present tense
- Keep it under 72 characters
- Be specific about what changed

4.11 Practical Exercise — Build a Mini Project

Follow these steps exactly. Do not skip ahead.

1. Create a new folder called `practice-repo` on your Desktop

2. Initialise Git inside it

3. Create a file called `index.html` with this content:

My first webpage

4. Check status — what do you see?

5. Stage `index.html`

6. Check status again — what changed?

7. Commit with the message `"Add homepage"`

8. Add a second line to `index.html` :

Built with Git

9. Run `git diff` — read the output carefully

10. Stage and commit with message `"Add subtitle to homepage"`

11. Create a file called `passwords.txt` with content `secret123`

12. Create a `.gitignore` file that ignores `passwords.txt`

13. Run `git status` — confirm `passwords.txt` does not appear

14. Commit the `.gitignore`

15. Run `git log --oneline` — you should see 3 commits

Chapter Summary

Command	What it does
`git init`	Initialise a new repository
`git status`	Show current state of all three zones
`git add filename`	Stage a specific file
`git add .`	Stage all changed files
`git add -p`	Stage changes interactively
`git diff`	Show unstaged changes
`git diff --staged`	Show staged changes
`git commit -m "msg"`	Save staged changes to history
`git log --oneline`	Show compact commit history
`git restore filename`	Discard working tree changes
`git restore --staged`	Unstage a file
`git commit --amend`	Fix last commit message
`git rm --cached`	Stop tracking a file

Assessment — Test Yourself

Question 1 You have modified three files — `index.html`, `style.css` and `script.js`. You only want to commit the changes to `index.html` right now. What commands do you run?

Question 2 You run `git status` and see:

Changes not staged for commit: modified: readme.txt

What does this mean exactly and what are your options?

Question 3 What is the difference between `git diff` and `git diff --staged`?

Question 4 You accidentally typed the wrong commit message. The commit has not been pushed yet. How do you fix it?

Question 5 You created a file called `api-keys.txt` and accidentally committed it. You then added it to `.gitignore`. Is it now safe? Explain your answer and describe what you should do.

Question 6 What does the `>>` operator do that `>` does not? Why does this matter when adding content to a file?

Answers

Answer 1

git add index.html git commit -m "Your message about index.html changes"

Only `index.html` is staged and committed. The changes to `style.css` and `script.js` remain in the working tree, unaffected.

Answer 2 It means `readme.txt` is a file Git is already tracking (it has been committed before) and changes have been made to it since the last commit. Those changes are in Zone 1 (working tree) but have not been moved to Zone 2 (staging area) yet. Options:

- Stage it: `git add readme.txt` then commit
- Discard the changes: `git restore readme.txt`
- Leave it as is and work on something else first

Answer 3 `git diff` shows changes between the working tree and the staging area — changes you have made but not yet staged. `git diff --staged` shows changes between the staging area and the last commit — changes you have staged but not yet committed. Use `git diff --staged` as a final check before committing to confirm exactly what will be saved.

Answer 4

git commit --amend -m "Corrected commit message"

This replaces the last commit message. Only use this before pushing. Once a commit is pushed to a shared branch, amending it rewrites history and causes problems for anyone who has already pulled that commit.

Answer 5 No — it is NOT safe. Adding a file to `.gitignore` after it has already been committed does not remove it from Git history. The file and its contents are permanently recorded in past commits. Anyone who clones the repo can see the historical commits and read the file contents. What you should do:

1. `git rm --cached api-keys.txt` — stop tracking the file
2. `git commit -m "Remove api-keys.txt from tracking"`
3. Consider rotating (changing) the exposed API keys immediately
4. If the repo is public, assume the keys are compromised

Answer 6 `>` creates a new file or completely overwrites an existing file. `>>` appends to the end of a file without destroying existing content. This matters because using `>` on an existing file destroys all previous content — a dangerous mistake if you intended to add a line. Always use `>>` when adding to existing files from the terminal.

What is Next

In Chapter 5 we explore one of Git's most powerful features — branching. You will create parallel timelines, work independently on features, and merge your work back together.

Proceed to Chapter 5 — Branching

Chapter 5 — Branching

Learning Objectives

By the end of this chapter you will be able to:

- Explain what a branch is and how it works internally
- Create and switch between branches
- Understand what happens to your files when you switch branches
- Merge branches using fast-forward and 3-way merge
- Identify, understand and resolve merge conflicts
- Delete branches safely after merging
- Visualise your branch history with git log --graph

5.1 What is a Branch?

A branch is one of Git's most powerful and most misunderstood features.

The common misconception

Most beginners imagine a branch as a copy of the project in a separate folder — like saving a duplicate of your files with a different name. This is wrong.

What a branch actually is

A branch is simply a **lightweight pointer to a commit**.

That is all. A label. A sticky note attached to a specific commit. When you make a new commit on a branch, the pointer moves forward automatically to point at the new commit.

Before new commit: After new commit:

C1 --> C2 --> C3 C1 --> C2 --> C3 --> C4 ^ ^ main main

This is why creating a branch in Git is nearly instant — Git is not copying any files. It is just creating a new pointer.

Why branches matter

Branches let you:

- Work on a new feature without touching the main codebase
- Fix a bug while someone else builds a feature simultaneously
- Experiment freely — if it fails, just delete the branch
- Review changes before they go into the main codebase

5.2 The Default Branch

When you run `git init`, Git creates a default branch.

In modern Git the default branch is called `main`. In older repos you may see `master` — they work identically, only the name is different.

HEAD and branches

Remember HEAD from Chapter 2? It is the pointer that shows where you are right now.

C1 --> C2 --> C3 ^ main <-- HEAD

HEAD points to main. Main points to C3. You are currently at C3 on the main branch.

When you create a new branch, HEAD can move to point at it:

C1 --> C2 --> C3 ^ main ^ feature <-- HEAD

5.3 Creating and Switching Branches

See all branches

git branch

The branch with `*` is your current branch: main

Create a new branch

git branch feature-login

This creates the branch but does NOT switch to it. You are still on main.

git branch

feature-login

main

Switch to the new branch

Modern way (Git 2.23+):

git switch feature-login

Classic way (still works):

git checkout feature-login

After switching:

git branch

feature-login main

The `*` has moved to `feature-login`. You are now on the new branch.

Create and switch in one command

This is what you will use most often:

git switch -c feature-login

The `-c` flag means "create". This creates the branch AND switches to it in a single command.

5.4 Working on a Branch

Once you are on a branch, every commit you make belongs to that branch. The main branch is completely unaffected.

Practical example

Start on main with 3 commits:

git log --oneline

c3f9a2b (HEAD -> main) Add about page b8d2f1a Add homepage a3f9c2b Add readme

Create and switch to a feature branch:

git switch -c feature-contact-page

Create a new file:

echo Contact us at info@example.com > contact.txt git add contact.txt git commit -m "Add contact page"

View the log:

git log --oneline

d4e5f6a (HEAD -> feature-contact-page) Add contact page c3f9a2b (main) Add about page b8d2f1a Add homepage a3f9c2b Add readme

Notice:

- `HEAD -> feature-contact-page` — you are on the feature branch
- `main` is still at `c3f9a2b` — completely untouched
- The feature branch has one commit that main does not

What happens when you switch back to main

git switch main

Now open your file explorer and look at the project folder. `contact.txt` has disappeared.

This is not a bug. Git has restored your working tree to match exactly what main looks like. The file exists on the feature branch — it simply does not belong to main yet.

Switch back to the feature branch:

git switch feature-contact-page

`contact.txt` reappears. This is Git managing your working tree automatically as you move between branches.

5.5 Merging Branches

When the work on a branch is complete and tested, you merge it into main. There are two types of merge.

Type 1 — Fast-forward merge

A fast-forward merge happens when main has no new commits since the branch was created. Git simply moves the main pointer forward.

Before merge: After merge:

C1 --> C2 --> C3 --> C4 C1 --> C2 --> C3 --> C4 ^ ^ ^ main feature main, feature

No new commit is created. Git just moves the label.

How to do it:

First switch to the branch you want to merge INTO:

git switch main

Then merge:

git merge feature-contact-page

Output:

Updating c3f9a2b..d4e5f6a Fast-forward contact.txt | 1 + 1 file changed, 1 insertion(+) create mode 100644 contact.txt

The word `Fast-forward` confirms this type of merge happened.

Type 2 — 3-way merge

A 3-way merge happens when both main and the feature branch have new commits since the branch was created. Git cannot simply move a pointer — it must create a new merge commit that combines both.

Before merge:

C1 --> C2 --> C3 --> C4 (main)

C5 --> C6 (feature)

After merge:

C1 --> C2 --> C3 --> C4 -----> M (merge commit) \ / C5 --> C6 ---

Git uses three points to calculate the merge:

1. The common ancestor (C3 — where the branch started)
2. The tip of main (C4)
3. The tip of the feature branch (C6)

How to do it:

git switch main git merge feature-branch

Git opens your editor for a merge commit message. Accept the default message and save.

5.6 Merge Conflicts

A merge conflict occurs when two branches have made different changes to the same lines of the same file. Git cannot decide which version is correct — it stops and asks you to decide.

How to create a conflict (for practice)

Step 1 — Start on main and edit a file:

echo Git is a version control system > readme.txt git add readme.txt git commit -m "Update readme on main"

Step 2 — Create a branch and make a different edit to the same line:

git switch -c fix-readme echo Git is a powerful version control tool > readme.txt git add readme.txt git commit -m "Update readme on fix-readme branch"

Step 3 — Switch to main and merge:

git switch main git merge fix-readme

Git stops with:

Auto-merging readme.txt CONFLICT (content): Merge conflict in readme.txt Automatic merge failed; fix conflicts and then commit the result.

Reading the conflict markers

Open `readme.txt` and you will see: <<<<<<< HEAD Git is a version control system

Git is a powerful version control tool

fix-readme

Understanding each part:

Marker	Meaning
`<<<<<<< HEAD`	Start of YOUR version (current branch)
`=======`	Dividing line between the two versions
`>>>>>>> fix-readme`	End of THEIR version (branch being merged)

Resolving the conflict

You have three choices:

1. Keep your version (delete their version and the markers)
2. Keep their version (delete your version and the markers)
3. Write something new that combines both

For this example, write a combined version:

Edit the file so it contains only this:

Git is a powerful version control system

Remove all three marker lines completely. The file must contain only the content you want — no `<<<<<<<`, `=======` or `>>>>>>>`.

Completing the merge

After editing:

git add readme.txt git commit -m "Resolve merge conflict in readme"

The merge is complete.

Checking status during a conflict

git status

Output during a conflict:

On branch main You have unmerged paths. (fix conflicts and run "git commit")

Unmerged paths: (use "git add ..." to mark resolution) both modified: readme.txt

`both modified` means both branches changed the same file.

Aborting a merge

If you want to cancel the merge and go back to where you were:

git merge --abort

This restores your working tree to the state before the merge started.

5.7 Visualising Branch History

The most useful command for understanding your branch history:

git log --oneline --graph --all

Example output: 1def4a8 (HEAD -> main) Resolve merge conflict in readme | | * daf02c2 (fix-readme) Update readme on fix-readme branch | 53838be Update readme on main |/ debfd04 Add contact page c3f9a2b Add about page a3f9c2b Add readme

Reading this graph:

Symbol	Meaning
*	A commit
|	A branch line
|\	Branch splitting apart
|/	Branch merging back together

Set up a short alias for this command:

git config --global alias.lg "log --oneline --graph --all"

Then just type `git lg` to see the full visual history.

5.8 Deleting Branches

After a branch has been merged, it has served its purpose. Delete it to keep your branch list clean.

Delete a merged branch

git branch -d feature-contact-page

Git checks that the branch has been fully merged before deleting. If it has not been merged, Git refuses and shows a warning.

Force delete an unmerged branch

git branch -D feature-contact-page

Capital `-D` skips the merge check and deletes regardless. Use this when you want to abandon work on a branch entirely.

View all branches including deleted ones

Deleted branches are gone locally but their commits remain in `git reflog` for 90 days. If you accidentally delete a branch you can recover it:

git reflog

Find the commit SHA of the branch tip, then:

git switch -c recovered-branch a3f9c2b

5.9 Branch Naming Conventions

Good branch names make history readable and teams efficient.

Common naming patterns

Pattern	Example	Used for
`feature/`	`feature/user-login`	New features
`fix/`	`fix/navbar-bug`	Bug fixes
`hotfix/`	`hotfix/payment-error`	Urgent production fixes
`docs/`	`docs/update-readme`	Documentation only
`refactor/`	`refactor/clean-database`	Code restructuring

Rules for branch names

- Use lowercase letters
- Use hyphens not spaces (`feature-login` not `feature login`)
- Be descriptive but concise
- Avoid special characters except `/` and `-`

5.10 Common Branching Workflows

GitHub Flow (recommended for most teams)

Create a branch from main Make commits on the branch Open a pull request Review and discuss Merge into main Delete the branch

Simple, clean, works for teams of any size.

Personal workflow (solo developer)

Create a branch for each feature or fix Work on the branch Merge back to main when done Delete the branch

---

Chapter Summary

Command	What it does
`git branch`	List all branches
`git branch name`	Create a new branch
`git switch name`	Switch to a branch
`git switch -c name`	Create and switch in one step
`git merge name`	Merge a branch into current branch
`git branch -d name`	Delete a merged branch
`git branch -D name`	Force delete any branch
`git merge --abort`	Cancel a merge in progress
`git log --oneline --graph --all`	Visual branch history

Assessment — Test Yourself

Question 1 What is a branch in Git? Correct the following statement: "A branch is a copy of the project files in a separate folder."

Question 2 You are on `main` and run `git branch feature-x`. What is the state of your branches now? Where is HEAD pointing?

Question 3 What is the difference between a fast-forward merge and a 3-way merge? When does each occur?

Question 4 You open a file during a merge conflict and see: <<<<<<< HEAD colour = blue

colour = green

design-branch

Explain what each section means and describe three ways you could resolve this conflict.

Question 5 You run `git branch -d old-feature` and Git refuses, saying the branch is not fully merged. What are your options and what should you consider before proceeding?

Question 6 Why does `contact.txt` disappear from your folder when you switch from `feature-contact-page` back to `main`? Is the file lost?

Answers

Answer 1 A branch is a lightweight pointer to a commit — it is just a label that moves forward as new commits are made. It is NOT a copy of files. Creating a branch takes milliseconds and almost no disk space because Git only

creates a new pointer, not new files. The corrected statement: "A branch is a lightweight pointer to a specific commit that allows independent lines of development without copying any files."

Answer 2 After running `git branch feature-x`:

- Two branches exist: `main` and `feature-x`
- Both point to the same commit
- HEAD still points to `main` — you have NOT switched branches
- The `*` in `git branch` output is still next to `main` You must run `git switch feature-x` to move to the new branch.

Answer 3 A fast-forward merge occurs when the base branch (main) has no new commits since the feature branch was created. Git simply moves the main pointer forward to the tip of the feature branch. No new commit is created. A 3-way merge occurs when both branches have new commits since they diverged. Git cannot simply move a pointer — it must calculate the differences between both branches and their common ancestor, then create a new merge commit that combines them.

Answer 4

- `<<<<<<< HEAD` to `=======`: the current branch version — `colour = blue`
- `=======` to `>>>>>>>`: the incoming branch version — `colour = green`
- `>>>>>>> design-branch`: end of the conflict block

Three ways to resolve:

1. Keep HEAD version: delete everything except `colour = blue`
2. Keep incoming version: delete everything except `colour = green`
3. Combine: write `colour = blue-green` or any other resolution In all cases remove the three marker lines completely, then `git add` the file and `git commit`.

Answer 5 Options:

1. Merge the branch first: `git merge old-feature` then delete with `-d`
2. Force delete: `git branch -D old-feature` — permanently loses any unmerged commits

Before force deleting, consider:

- Does this branch contain work worth keeping?
- Has anyone else worked on this branch?
- Can the commits be cherry-picked to another branch first? If the work is truly unwanted, force delete is safe. Remember that `git reflog` can recover the commits for up to 90 days if you change your mind.

Answer 6 `contact.txt` has not been lost. Git manages your working tree automatically when you switch branches. When you switch to `main`, Git restores your folder to exactly match the state of main — which does not include `contact.txt` because that file was only committed on `feature-contact-page`.

Switch back to `feature-contact-page` and `contact.txt` reappears instantly. The file is safe in Git's history. It will appear in your folder on any branch that contains the commit that added it.

What is Next

In Chapter 6 we connect your local Git repository to the world. You will create a GitHub account, push your project online, and learn to collaborate with others through remotes.

Proceed to Chapter 6 — GitHub and Remotes

Chapter 6 — GitHub and Remotes

Learning Objectives

By the end of this chapter you will be able to:

- Explain the difference between Git and GitHub clearly
- Create a GitHub account and set up your profile
- Create a remote repository on GitHub
- Connect your local repository to GitHub
- Push your commits to GitHub
- Pull changes from GitHub to your local machine
- Clone any repository from GitHub
- Understand the difference between fetch and pull
- Use personal access tokens for authentication

6.1 From Local to Global

So far everything you have done has lived on your own machine. Your commits, your branches, your history — all local.

This chapter connects your local Git to the world.

Before this chapter: After this chapter:

Your machine only Your machine <--> GitHub (the internet)

GitHub gives you:

- A backup of your entire project history in the cloud
- A URL you can share with anyone
- A platform for collaboration with other developers
- A public portfolio of your work
- Free website hosting (GitHub Pages)

6.2 Git vs GitHub — Final Clarification

By now you know the difference, but it is worth stating one final time clearly:

Git GitHub

Software on your computer Website on the internet Tracks changes locally Hosts repositories online Works without internet Needs internet to access Free, open source Free tier + paid plans Created by Linus Torvalds Owned by Microsoft

You use Git to manage your code. You use GitHub to share it.

6.3 Creating a GitHub Account

Step 1 — Go to GitHub

https://github.com

Step 2 — Sign up

Click **Sign up** and provide:

- A valid email address
- A password
- A username

Choosing your username: Your username becomes part of every repo URL you create:

https://github.com/YOUR-USERNAME/repo-name

Choose something professional — your name or a variation of it. This will appear on your public profile and in your commit history.

Step 3 — Verify your email

GitHub sends a verification email. Click the link to activate your account.

Step 4 — Complete your profile

Go to your profile settings and add:

- Your full name
- A professional photo
- A bio describing what you do
- Your website or portfolio URL
- Your location and company

A complete profile looks professional and builds credibility.

6.4 Creating a Repository on GitHub

Step 1 — Click the + icon

In the top right corner of GitHub, click `+` then **New repository**.

Step 2 — Fill in the details

Field	What to enter
Repository name	Short, descriptive, use hyphens not spaces
Description	One sentence explaining what this repo is
Public / Private	Public for portfolio work, Private for sensitive projects
Initialize with README	Leave UNCHECKED if you have an existing local repo
Add .gitignore	Leave as None if you have an existing local repo
Choose a license	Optional — MIT is the most common open source licence

Step 3 — Click Create repository

GitHub creates an empty repository and shows you setup instructions.

> *Important: If you already have a local repository with commits, do NOT check "Initialize with README". An initialised GitHub repo and your local repo will conflict when you try to push.*

6.5 Connecting Local to Remote

After creating an empty GitHub repo, connect your local repo to it.

The remote add command

git remote add origin https://github.com/USERNAME/repo-name.git

Breaking this down:

Part	Meaning
`git remote add`	Add a new remote connection
`origin`	The name for this remote (conventional name)
`https://...`	The URL of the GitHub repository

Verify the connection

git remote -v

Output:

origin https://github.com/USERNAME/repo-name.git (fetch) origin https://github.com/USERNAME/repo-name.git (push)

Two lines appear — one for fetching (downloading) and one for pushing (uploading). Both point to the same URL.

6.6 Pushing to GitHub

Your first push

git push -u origin main

Breaking this down:

Part	Meaning
`git push`	Upload commits to the remote
`-u`	Set up tracking — remember this remote for future pushes
`origin`	The remote to push to
`main`	The branch to push

The `-u` flag is only needed the first time. After that, simply type:

git push

Git remembers where to push.

Authentication

When you push for the first time, GitHub asks you to authenticate.

Option 1 — Git Credential Manager (Windows) A window appears asking for your GitHub username and password. Use your GitHub password here — or sign in via browser.

Option 2 — Personal Access Token GitHub no longer accepts your account password directly in the terminal on some systems. Instead you use a token.

To create a token:

GitHub → Profile photo → Settings → Developer settings → Personal access tokens → Tokens (classic) → Generate new token (classic)

Settings:

- Note: give it a descriptive name
- Expiration: 90 days
- Scopes: tick the `repo` checkbox

Click **Generate token**. Copy it immediately — GitHub shows it only once.

When Git asks for your password in the terminal, paste the token.

What happens after pushing

Visit your GitHub repo URL and refresh. You will see:

- All your files listed
- Your commit messages next to each file
- Your README rendered at the bottom
- Your commit count
- The time of your last commit

6.7 Understanding Remotes

What is origin?

`origin` is just a name — a shortcut for the full GitHub URL. Instead of typing the full URL every time you push or pull, you use the name `origin`.

You could call it anything:

git remote add github https://github.com/... git remote add myrepo https://github.com/...

But `origin` is the universal convention. Every Git tutorial, every team, every company uses `origin` for the primary remote. Stick with it.

Multiple remotes

You can have more than one remote:

git remote add origin https://github.com/USERNAME/repo.git git remote add backup https://gitlab.com/USERNAME/repo.git

Push to a specific remote:

git push origin main git push backup main

This is useful for mirroring a repo to multiple platforms.

View all remotes

git remote -v

Remove a remote

git remote remove origin

Rename a remote

git remote rename origin github

6.8 Pulling from GitHub

When changes exist on GitHub that you do not have locally, you need to download them.

git pull

git pull

This does two things in one step:

1. Downloads new commits from GitHub (fetch)
2. Merges them into your current branch (merge)

If there are no conflicts, the merge happens automatically.

git fetch

git fetch

This only downloads the changes — it does NOT merge them. Your working files are unchanged. You can review what arrived before deciding to merge.

To see what was fetched:

git log origin/main --oneline

To merge after fetching:

git merge origin/main

fetch vs pull — when to use each

Situation	Use
Trust the incoming changes, want them now	`git pull`
Want to review changes before merging	`git fetch` then inspect
Working in a team with frequent changes	`git fetch` first
Solo project, your own remote	`git pull` is fine

The professional habit is `git fetch` first, check what changed, then `git merge`. But `git pull` is perfectly acceptable for most situations.

6.9 Cloning a Repository

Cloning downloads a complete copy of any repository — including all commits, all branches and all history.

Clone a public repo

git clone https://github.com/USERNAME/repo-name.git

This creates a new folder called `repo-name` on your machine with everything inside it.

Clone to a specific folder name

git clone https://github.com/USERNAME/repo-name.git my-folder

What clone does automatically

Action	What Git does
Downloads all files	Complete working tree
Downloads all history	Every commit ever made
Sets up origin	Points to the cloned URL
Sets up tracking	Local main tracks origin/main

After cloning you can immediately start working — `git add`, `git commit`, `git push`.

Cloning vs Forking

	Clone	Fork
Where	Creates local copy	Creates GitHub copy
Push access	Only if you own the repo	Always — it is your fork
Use case	Work on your own repos	Contribute to others' repos

6.10 Tracking Branches

When you push with `-u` or clone a repo, Git sets up **tracking branches** — a link between your local branch and the remote branch.

git push -u origin main

After this:

- Local `main` tracks `origin/main`
- `git push` knows to push to `origin/main`
- `git pull` knows to pull from `origin/main`

Seeing tracking relationships

git branch -vv

Output: main a3f9c2b [origin/main] Add readme file

`[origin/main]` shows which remote branch `main` is tracking.

6.11 The Complete Remote Workflow

Starting a new project

Step 1: Create repo on GitHub (empty, no README) Step 2: Create local folder and initialise Git mkdir my-project cd my-project git init Step 3: Create files and make commits git add . git commit -m "Initial commit" Step 4: Connect to GitHub git remote add origin https://github.com/USERNAME/my-project.git Step 5: Push git push -u origin main

Daily workflow with a remote

Morning — get latest changes: git pull

During the day — work normally: git add . git commit -m "What I did"

End of day — push your work: git push

Joining an existing project

Step 1: Clone the repo git clone https://github.com/USERNAME/project.git Step 2: Work on a branch git switch -c my-feature Step 3: Commit your work git add . git commit -m "Add my feature" Step 4: Push your branch git push origin my-feature Step 5: Open a pull request on GitHub

---

6.12 Common Remote Problems and Fixes

Problem: Push rejected

! [rejected] main -> main (fetch first) error: failed to push some refs

Cause: Someone else pushed to the remote since your last pull. Your local history is behind the remote.

Fix:

git pull git push

Problem: Authentication failed

remote: Invalid username or password

Fix: Use a Personal Access Token instead of your password. See section 6.6 for how to create one.

Problem: Remote origin already exists

error: remote origin already exists

Fix: Remove the existing remote and add the correct one:

git remote remove origin git remote add origin https://github.com/USERNAME/repo.git

Problem: Repository not found

ERROR: Repository not found

Cause: Wrong URL, or the repo is private and you are not authenticated.

Fix: Check the URL carefully. Ensure you are logged in and have access to the repo.

Chapter Summary

Command	What it does
`git remote add origin URL`	Connect local repo to GitHub
`git remote -v`	Show all remote connections
`git remote remove name`	Remove a remote connection
`git push -u origin main`	First push — sets up tracking
`git push`	Push commits to tracked remote
`git pull`	Download and merge remote changes
`git fetch`	Download remote changes without merging
`git clone URL`	Download a complete repo from GitHub
`git branch -vv`	Show tracking branch relationships

Assessment — Test Yourself

Question 1 What is the difference between `git push` and `git push -u origin main`? When do you use each?

Question 2 You run `git pull` and Git tells you there is a merge conflict. What happened and what do you do next?

Question 3 What is the difference between `git fetch` and `git pull`? Describe a situation where you would prefer `git fetch`.

Question 4 Your colleague emails you a GitHub repository link and asks you to work on it. You do not have push access to their repo. What is the correct workflow?

Question 5 You run `git push` and get the error:

! [rejected] main -> main (fetch first)

What caused this error and how do you fix it?

Question 6 Why should you NOT check "Initialize with README" on GitHub when you already have a local repository with commits?

Answers

Answer 1 `git push -u origin main` pushes your commits to the `main` branch on the `origin` remote AND sets up tracking — telling Git that your local `main` branch should track `origin/main` for future pushes and pulls. Use this only the first time you push a branch.

After the first push, simply use `git push`. Git already knows the tracking relationship and pushes to the correct remote and branch automatically.

Answer 2 A merge conflict during `git pull` means the remote has changes to the same lines of the same files that you have also changed locally. Git cannot automatically merge them.

What to do:

1. Open the conflicted files — look for `<<<<<<<`, `=======`, `>>>>>>>` markers
2. Edit each file to resolve the conflict — keep the correct version
3. Remove all conflict markers
4. Stage the resolved files: `git add filename`
5. Complete the merge: `git commit`

Answer 3 `git fetch` downloads new commits from the remote but leaves your working files and current branch completely unchanged. `git pull` downloads AND immediately merges the remote changes into your current branch — it is `git fetch` + `git merge` combined.

Prefer `git fetch` when:

- Working in a team and you want to review incoming changes first
- You are mid-way through work and not ready to merge
- You want to compare remote changes against your local work before deciding to integrate

Answer 4 The correct workflow is:

1. Fork the repository — creates your own copy on GitHub
2. Clone YOUR fork to your local machine
3. Create a branch for your changes
4. Make commits on your branch
5. Push your branch to YOUR fork
6. Open a pull request from your fork to the original repo
7. The repo owner reviews and merges your contribution

Answer 5 This error means the remote branch has commits that your local branch does not have. Someone else pushed changes after your last pull, so your local history is behind the remote.

Fix:

git pull

This downloads and merges the remote changes. If there are no conflicts, then:

git push

If there are conflicts, resolve them first then push.

Answer 6 If you check "Initialize with README", GitHub creates an initial commit in the remote repo. Your local repo also has commits. These two histories have no common ancestor — Git cannot merge them.

When you try to push you get:

! [rejected] main -> main (non-fast-forward)

To avoid this entirely, always create an empty GitHub repo (no README, no .gitignore, no licence) when connecting to an existing local repo.

What is Next

In Chapter 7 we explore the most important collaboration feature on GitHub — Pull Requests. You will learn how teams review, discuss and safely merge code changes.

Proceed to Chapter 7 — Collaboration and Pull Requests

Chapter 7 — Collaboration and Pull Requests

Learning Objectives

By the end of this chapter you will be able to:

- Explain what a pull request is and why teams use them
- Open a pull request on GitHub
- Review and comment on a pull request
- Request changes and approve a pull request
- Merge a pull request using different strategies
- Understand GitHub Flow as a team workflow
- Fork a repository and contribute to open source
- Set up branch protection rules
- Close and delete branches after merging

7.1 Why Pull Requests Exist

In Chapter 5 you learned to merge branches locally on your own machine. That works perfectly when you are working alone.

But in a team, merging directly is dangerous:

- No one reviews the changes before they go live
- Broken code can reach the main branch
- There is no discussion or documentation of why changes were made
- Junior developers can accidentally overwrite important code

The solution is the **Pull Request** — GitHub's most important collaboration feature.

A pull request is a formal proposal to merge one branch into another. It creates a space for:

- Code review
- Discussion and comments
- Automated testing
- Approval before merging

Nothing reaches `main` without going through this process.

7.2 What is a Pull Request?

A pull request (PR) is NOT a Git command. It is a GitHub feature.

Without pull requests: With pull requests:

Branch --> merge --> main Branch --> PR --> Review --> Approve --> merge --> main (direct, no review) (reviewed, discussed, tested, approved)

When you open a pull request you are saying:

> *"I have made changes on this branch. Please review them, discuss them, test them, and if they look good — merge them into main."*

What a pull request contains

- The list of commits being proposed
- A diff showing exactly what lines changed
- A title and description explaining the changes
- A conversation thread for comments
- A status showing if automated checks pass or fail
- An approval system for reviewers

7.3 The GitHub Flow

GitHub Flow is the most widely used team workflow in the world. It is simple, clean and works for teams of any size.

Step 1: Create a branch from main git switch -c feature/my-feature

Step 2: Make commits on the branch git add . git commit -m "Add my feature"

Step 3: Push the branch to GitHub git push origin feature/my-feature

Step 4: Open a pull request on GitHub

Step 5: Team reviews the code Comments, suggestions, approvals

Step 6: Merge the pull request into main

Step 7: Delete the branch git branch -d feature/my-feature

Main is always deployable. Every change goes through review. No exceptions.

7.4 Opening a Pull Request

Step 1 — Push your branch

git switch -c feature/add-contact-page

Make your changes, commit them, then push:

git push origin feature/add-contact-page

Step 2 — Go to GitHub

After pushing, GitHub shows a yellow banner:

feature/add-contact-page had recent pushes — Compare & pull request

Click **Compare & pull request**.

Alternatively go to the **Pull requests** tab and click **New pull request**.

Step 3 — Fill in the PR form

Title: A clear, concise description of what changed

Add contact page with email form

Description: Explain what you did and why. Use this template: What this PR does

Adds a contact page with a working email form.

Why

Users need a way to get in touch. Currently there is no contact information on the site.

How to test Navigate to /contact Fill in the form Submit — you should see a confirmation message Screenshots

[attach before/after screenshots if relevant]

Reviewers: Tag teammates who should review this

Labels: Tag the type of change (feature, bug fix, docs)

Step 4 — Click Create pull request

The PR is now open. Reviewers are notified.

7.5 Reviewing a Pull Request

As a reviewer your job is to ensure the code is correct, readable and safe before it merges into main.

What to look for

Area	Questions to ask
Correctness	Does the code do what the PR description says?
Logic	Are there any bugs or edge cases not handled?
Readability	Is the code clear and well named?
Tests	Are there tests for the new functionality?
Security	Does this introduce any security risks?
Style	Does it follow the team's coding conventions?

Leaving comments

Click on any line in the **Files changed** tab to leave an inline comment on that specific line.

Types of feedback:

Type	When to use
Praise	Something done particularly well
Question	You need clarification before approving
Suggestion	An improvement that is not blocking
Request changes	Something must be fixed before merging

Submitting your review

After reviewing click **Review changes** and choose:

- **Comment** — general feedback, no formal approval
- **Approve** — you are happy with the changes, ready to merge
- **Request changes** — changes must be made before merging

7.6 Responding to Review Feedback

As the author of a pull request, when a reviewer requests changes:

Step 1 — Read the feedback carefully

Understand exactly what the reviewer wants changed. If unclear, ask for clarification in the comment thread.

Step 2 — Make the changes locally

git switch feature/add-contact-page

Make the requested changes, then commit:

git add . git commit -m "Address review feedback - improve form validation"

Step 3 — Push the update

git push origin feature/add-contact-page

The pull request automatically updates with the new commits. You do not need to open a new PR.

Step 4 — Reply to the reviewer

In the PR comment thread, reply to let the reviewer know you have addressed their feedback. Tag them with `@username`.

Step 5 — Re-review

The reviewer checks the changes and approves if satisfied.

7.7 Merging a Pull Request

Once approved, the PR is ready to merge. GitHub offers three merge strategies:

Strategy 1 — Create a merge commit

Creates a merge commit preserving full history

Keeps a complete record of the branch and when it was merged. The history shows the branching clearly.

Best for: Teams that want full history preservation.

Strategy 2 — Squash and merge

Combines all commits into one before merging

If your branch has 15 messy "work in progress" commits, squash combines them all into one clean commit on main.

Best for: Keeping main history clean and readable.

Strategy 3 — Rebase and merge

Replays commits on top of main — linear history

Creates a linear history without a merge commit.

Best for: Teams that prefer linear history.

After merging

GitHub shows:

Pull request successfully merged and closed

Click **Delete branch** to remove the feature branch from GitHub.

Then locally:

git switch main git pull git branch -d feature/add-contact-page

This syncs your local main with the merged changes and cleans up the local branch.

7.8 Draft Pull Requests

A draft pull request signals that the work is in progress and not ready for review yet.

Use a draft PR when:

- You want early feedback on direction before completing the work
- You want automated tests to run while you continue working
- You are working on a long feature and want visibility

To open a draft PR:

- Click the dropdown arrow next to **Create pull request**
- Select **Create draft pull request**

When ready for review, click **Ready for review** to convert it to a regular PR.

7.9 Forking and Contributing to Open Source

Forking allows you to contribute to any public repository on GitHub even without write access to the original.

The open source contribution workflow

Step 1 — Fork the repository

On the original repo's GitHub page, click **Fork**. GitHub creates your own copy at:

https://github.com/YOUR-USERNAME/original-repo-name

Step 2 — Clone YOUR fork

git clone https://github.com/YOUR-USERNAME/original-repo-name.git cd original-repo-name

Step 3 — Add the original as upstream

git remote add upstream https://github.com/ORIGINAL-OWNER/original-repo-name.git

Now you have two remotes:

origin — your fork (you can push here) upstream — the original repo (you can only pull from here)

Step 4 — Create a branch

git switch -c fix/spelling-error

Step 5 — Make your changes and commit

git add . git commit -m "Fix spelling error in README"

Step 6 — Push to YOUR fork

git push origin fix/spelling-error

Step 7 — Open a pull request

Go to the ORIGINAL repo on GitHub. GitHub detects your recent push and offers:

Compare & pull request

Open the PR from your fork's branch to the original repo's main.

Step 8 — Wait for review

The original repo's maintainers review your PR. They may:

- Merge it as is
- Request changes
- Ask questions
- Close it if it does not fit the project

Keeping your fork up to date

While you work, the original repo continues to get new commits. Keep your fork in sync:

git fetch upstream git switch main git merge upstream/main git push origin main

---

7.10 Branch Protection Rules

Branch protection rules prevent accidental or unauthorised changes to important branches like `main`.

Setting up branch protection

On GitHub:

Repository → Settings → Branches → Add branch protection rule

Branch name pattern: `main`

Common protection rules:

Rule	What it does
Require pull request before merging	Nobody can push directly to main
Require approvals	PR needs X approvals before merging

Require status checks to pass	CI tests must pass before merging
Require linear history	No merge commits — squash or rebase only
Include administrators	Even repo owners must follow the rules

Why branch protection matters

Without protection:

- Anyone can push broken code directly to main
- One mistake can break the entire application
- There is no audit trail of who approved what

With protection:

- Every change is reviewed before merging
- Automated tests must pass
- Main is always in a working state

7.11 GitHub Issues

Issues are GitHub's built-in task tracker. They work alongside pull requests to organise your project.

Creating an issue

Click **Issues** → **New issue**

A good issue contains:

- A clear title
- A description of the problem or feature request
- Steps to reproduce (for bugs)
- Expected vs actual behaviour (for bugs)
- Screenshots if relevant

Linking issues to pull requests

In your PR description, reference the issue:

Closes #42

When the PR merges, GitHub automatically closes issue #42.

Other keywords:

Fixes #42 Resolves #42

Issue labels

Organise issues with labels:

Label	Use
bug	Something is broken

`feature`	New functionality
`documentation`	Docs need updating
`good first issue`	Suitable for new contributors
`help wanted`	Maintainers need assistance

Chapter Summary

Concept	Key point
Pull request	A proposal to merge a branch — with review
GitHub Flow	Branch → commit → push → PR → review → merge
Code review	Check correctness, logic, readability, security
Merge strategies	Merge commit, squash, rebase — choose based on team preference
Draft PR	Work in progress — not ready for review yet
Fork	Your own copy of someone else's repo
Upstream	The original repo you forked from
Branch protection	Rules preventing direct pushes to important branches
Issues	GitHub's built-in task and bug tracker

Assessment — Test Yourself

Question 1 What is the difference between merging locally with `git merge` and merging via a GitHub pull request?

Question 2 Your team uses the squash and merge strategy. You made 8 commits on your feature branch before opening a PR. How many commits appear on main after the PR is merged?

Question 3 A reviewer leaves this comment on your PR: "This function will crash if the input is empty. Please add input validation." Describe the exact steps you take to respond.

Question 4 You want to contribute a bug fix to a popular open source project. You do not have write access to the repo. Describe the complete workflow from start to finish.

Question 5 Your team has set up branch protection requiring 2 approvals before merging. You are a repository administrator. Can you merge a PR with only 1 approval?

Question 6 What does writing `Closes #15` in a pull request description do?

Answers

Answer 1 `git merge` combines branches locally with no review process — changes go directly into the branch with no discussion or approval. A GitHub pull request creates a formal review process where teammates can examine every changed line, leave comments, request modifications, run automated tests and formally approve before

anything merges. Pull requests also create a permanent record of what was changed, who reviewed it, what was discussed and when it was merged — invaluable for team accountability.

Answer 2 One commit. Squash and merge combines all commits on the feature branch into a single commit before merging into main. The 8 individual commits disappear from main's history — only one clean commit representing the entire feature appears. The original commits still exist in the branch history and in `git reflog` but are not visible in main's linear history.

Answer 3 Step 1 — Read the comment carefully and understand exactly what validation is needed. Step 2 — Switch to the feature branch locally: `git switch feature/my-feature` Step 3 — Make the requested changes — add input validation Step 4 — Commit the fix: `git add .` `git commit -m "Add input validation to handle empty input"` Step 5 — Push the update: `git push origin feature/my-feature` Step 6 — Reply to the reviewer in the PR comment thread: "@reviewer — added input validation in the latest commit. The function now returns an error message if input is empty." Step 7 — Wait for the reviewer to check the changes and re-approve.

Answer 4

1. Go to the original repo on GitHub and click Fork
2. Clone YOUR fork: `git clone https://github.com/YOU/repo.git`
3. Add upstream: `git remote add upstream https://github.com/OWNER/repo.git`
4. Create a branch: `git switch -c fix/bug-description`
5. Make the fix and commit: `git commit -m "Fix bug description"`
6. Push to your fork: `git push origin fix/bug-description`
7. Go to the ORIGINAL repo on GitHub
8. Open a pull request from your fork's branch to original/main
9. Fill in the PR description explaining the fix
10. Wait for maintainers to review and respond

Answer 5 It depends on how the branch protection rule is configured. If the rule includes **Include administrators**, then no — even administrators must follow the rules and cannot merge with only 1 approval. If **Include administrators** is not checked, then yes — administrators can bypass the requirement. Best practice is to include administrators in the rules to ensure consistent standards across the entire team.

Answer 6 Writing `Closes #15` in a pull request description creates a link between the PR and issue number 15. When the PR is merged into the default branch, GitHub automatically closes issue 15 and shows the PR as the resolution. This keeps your project board clean and creates a clear connection between the problem (the issue) and the solution (the pull request).

What is Next

In Chapter 8 we learn the advanced Git commands that rescue you when things go wrong — stash, reset, revert, reflog and cherry-pick. These are the commands that separate confident Git users from beginners.

Proceed to Chapter 8 — Advanced Git — Rescue Commands

Chapter 8 — Advanced Git — Rescue Commands

Learning Objectives

By the end of this chapter you will be able to:

- Use git stash to shelve work in progress
- Understand the difference between git revert and git reset
- Use git reset with soft, mixed and hard options
- Recover lost commits using git reflog
- Apply specific commits with git cherry-pick
- Find which commit introduced a bug with git bisect
- Identify who changed a specific line with git blame
- Use interactive rebase to clean up commit history

8.1 Why These Commands Matter

The commands in this chapter are what separate confident Git users from beginners who panic when something goes wrong.

Every developer makes mistakes:

- Commits to the wrong branch
- Writes a bad commit message
- Accidentally deletes work
- Introduces a bug somewhere in 200 commits

These commands are your rescue toolkit. Learn them and nothing in Git will ever truly frighten you.

8.2 git stash — The Emergency Drawer

The problem it solves

You are mid-way through editing three files when your colleague says: "There is an urgent bug on main — can you fix it now?"

You cannot switch branches with uncommitted changes that would conflict. You are not ready to commit your half-finished work. What do you do?

You stash it.

How stash works

git stash

This takes all your uncommitted changes and saves them in a temporary storage area — completely separate from your commits. Your working tree becomes clean instantly. You can switch branches freely.

Before stash: After stash: readme.txt (modified) readme.txt (clean) style.css (modified) style.css (clean) new-page.html (untracked) new-page.html (still untracked)

Note: By default, stash saves tracked modified files. Untracked files are not stashed unless you use `-u`:

git stash -u

Retrieving your stash

After fixing the bug, get your work back:

git stash pop

Your changes reappear exactly as you left them.

Stash commands

Command	What it does
`git stash`	Save current changes to stash
`git stash -u`	Stash including untracked files
`git stash pop`	Restore latest stash and remove it
`git stash apply`	Restore latest stash but keep it in stash
`git stash list`	Show all stashes
`git stash drop`	Delete the latest stash
`git stash clear`	Delete all stashes

Multiple stashes

You can have several stashes at once:

git stash list

Output:

stash@{0}: WIP on main: a3f9c2b Add readme stash@{1}: WIP on feature: b8d2f1a Add homepage

Apply a specific stash:

git stash apply stash@{1}

Stash with a message

Give your stash a descriptive name:

git stash push -m "Half-finished login form"

8.3 git revert — Safe Undo

The problem it solves

You pushed a bad commit to a shared branch. Your teammates have already pulled it. How do you undo it without breaking their history?

How revert works

`git revert` creates a NEW commit that undoes the changes of a specific commit. It does NOT rewrite history — it adds to it.

Before revert: After revert:

C1 -> C2 -> C3 (bad) C1 -> C2 -> C3 (bad) -> C4 (reverts C3)

C3 is still in history. C4 cancels out its effects. Anyone who already has C3 can safely pull C4.

How to use it

Find the commit you want to undo:

git log --oneline

a3f9c2b (HEAD -> main) Bad commit - broke the login b8d2f1a Add user dashboard c4d5e6f Add homepage

Revert the bad commit:

git revert a3f9c2b

Git opens your editor for a revert commit message. Accept the default and save.

Output:

[main f7g8h9i] Revert "Bad commit - broke the login" 1 file changed, 1 deletion(-)

Reverting without opening the editor

git revert a3f9c2b --no-edit

Reverting multiple commits

git revert a3f9c2b b8d2f1a

This creates two revert commits — one for each.

When to use revert

Use `git revert` when:

- The commit has already been pushed to a shared branch
- You want to preserve the full history
- Teammates have already pulled the commit

8.4 git reset — Rewrite Local History

The problem it solves

You made several bad commits locally that you have NOT yet pushed. You want to erase them as if they never happened.

The golden rule of reset

NEVER use git reset on commits that have already been pushed to a shared branch. It rewrites history and causes serious problems for anyone who has pulled those commits.

The three modes of reset

`git reset` moves HEAD (and your branch pointer) to a previous commit. What happens to your changes depends on which mode you use.

git reset --soft COMMIT Commits erased, changes stay STAGED git reset --mixed COMMIT Commits erased, changes go to WORKING TREE git reset --hard COMMIT Commits erased, changes DELETED forever

Visualised:

Zone 1 Zone 2 Zone 3 Working tree Staging area Repository (commits)

--hard --mixed HEAD moves back changes changes to target commit deleted unstaged

```
        --soft
        changes
        stay staged
```

Practical example

You have these commits:

git log --oneline

d4e5f6a (HEAD -> main) Bad commit 3 c3b2a1f Bad commit 2 b8d2f1a Bad commit 1 a3f9c2b Good commit - last known good state

You want to go back to `a3f9c2b` and erase the three bad commits.

Using --soft (keep changes staged):

git reset --soft a3f9c2b

Result: Three commits erased. All changes are in the staging area. You can review them and re-commit differently.

Using --mixed (keep changes unstaged):

git reset --mixed a3f9c2b

Result: Three commits erased. All changes are in the working tree. You need to re-stage before committing.

Using --hard (delete everything):

git reset --hard a3f9c2b

Result: Three commits erased. ALL changes are permanently deleted. Your working tree matches exactly what `a3f9c2b` looked like.

Fixing just the last commit

If you only need to fix the most recent commit:

Fix the commit message:

git commit --amend -m "Corrected commit message"

Add a forgotten file to the last commit:

git add forgotten-file.txt git commit --amend --no-edit

8.5 git reflog — The Black Box Recorder

The problem it solves

You ran `git reset --hard` and deleted important work. Or you accidentally deleted a branch. The commits seem gone.

`git reflog` is your last safety net.

What reflog is

Reflog records every single movement of HEAD — every commit, every reset, every branch switch, every merge. It keeps this log for 90 days.

Even commits that appear deleted from `git log` are still visible in reflog.

How to use it

git reflog

Output:

a3f9c2b HEAD@{0}: reset: moving to a3f9c2b d4e5f6a HEAD@{1}: commit: Bad commit 3 c3b2a1f HEAD@{2}: commit: Bad commit 2 b8d2f1a HEAD@{3}: commit: Bad commit 1 a3f9c2b HEAD@{4}: commit: Good commit

Recovering lost commits

Find the SHA of the commit you want to recover — for example `d4e5f6a` — then:

git reset --hard d4e5f6a

Your lost commits are back.

Or create a new branch pointing to the lost commit:

git switch -c recovered-work d4e5f6a

Recovering a deleted branch

If you accidentally deleted a branch:

git reflog

Find the last commit that was on that branch, then:

git switch -c recovered-branch a3f9c2b

The branch is restored with all its commits.

8.6 git cherry-pick — Take One Commit

The problem it solves

A specific commit on one branch contains a fix you need on another branch — but you do not want to merge the entire branch.

How cherry-pick works

git cherry-pick COMMIT-SHA

This copies the changes from one specific commit and applies them as a new commit on your current branch.

Before cherry-pick: After cherry-pick:

main: C1 -> C2 main: C1 -> C2 -> C5' (copy of C5) feature: C1 -> C3 -> C4 -> C5

C5 is copied to main as C5' — same changes, new SHA hash.

Practical example

You are on main and need a bug fix from the feature branch:

git log feature --oneline

f9e8d7c Fix critical authentication bug a3f9c2b Add new dashboard feature b8d2f1a Start feature branch

Cherry-pick just the bug fix:

git cherry-pick f9e8d7c

Output:

[main g1h2i3j] Fix critical authentication bug 1 file changed, 3 insertions(+), 1 deletion(-)

Cherry-picking multiple commits

git cherry-pick f9e8d7c a3f9c2b

Cherry-picking a range of commits

git cherry-pick f9e8d7c..a3f9c2b

When cherry-pick conflicts

If the cherry-picked commit conflicts with your current branch:

1. Resolve the conflict as normal
2. `git add` the resolved files
3. `git cherry-pick --continue`

Or abort:

git cherry-pick --abort

8.7 git bisect — Find the Bug

The problem it solves

Your application worked 3 months ago but has a bug now. You have made 200 commits since then. Which commit introduced the bug?

`git bisect` uses binary search to find it in as few steps as possible.

How binary search works

Instead of checking all 200 commits one by one (worst case: 200 checks), bisect halves the search space each step. 200 commits = maximum 8 steps to find the bad commit.

200 commits -> 100 -> 50 -> 25 -> 13 -> 7 -> 4 -> 2 -> 1

How to use bisect

Step 1 — Start bisect:

git bisect start

Step 2 — Mark the current commit as bad:

git bisect bad

Step 3 — Mark a known good commit:

git bisect good a3f9c2b

Git checks out the commit halfway between good and bad.

Step 4 — Test and mark:

Test the application. Is the bug present?

If yes (bug present):

git bisect bad

If no (working correctly):

git bisect good

Git moves to the next midpoint. Repeat until Git says:

a3f9c2b is the first bad commit commit a3f9c2b Author: ... Date: ... Add payment processing feature

Step 5 — End bisect:

git bisect reset

This returns you to your original branch and commit.

8.8 git blame — Who Changed This Line?

The problem it solves

You found a bug in a specific line of code. You need to know who wrote it, when, and in which commit — so you can understand why it was written and how to fix it properly.

How to use blame

git blame filename.txt

Output:

a3f9c2b (Dr Hassan 2026-03-27 10:15:22 +0000 1) Git is a version control system b8d2f1a (John Smith 2026-03-28 14:30:45 +0000 2) Created by Linus Torvalds in 2005 c4d5e6f (Dr Hassan 2026-03-29 09:00:11 +0000 3) Used by over 100 million developers

Reading each column:

Column	Meaning
`a3f9c2b`	SHA of the commit that last changed this line
`Dr Hassan`	Author of that commit
`2026-03-27`	Date of that commit
`10:15:22`	Time of that commit
`1`	Line number
`Git is a...`	The actual line content

Blame for a specific section

git blame -L 10,25 filename.txt

Shows only lines 10 to 25.

Opening the commit from blame

Once you find the SHA, see the full commit:

git show a3f9c2b

This shows everything about that commit — the message, the author, the date, and every line that changed.

8.9 Interactive Rebase — Clean Up History

The problem it solves

You made 8 commits while working on a feature. Looking back, some have terrible messages, some are tiny fixups that should be combined with earlier commits, and one was committed in the wrong order.

Before opening a pull request you want the history to look clean and professional.

How interactive rebase works

git rebase -i HEAD~8

`HEAD~8` means "go back 8 commits from HEAD". This opens your editor showing the last 8 commits:

pick a3f9c2b Add login page pick b8d2f1a wip pick c4d5e6f fix typo pick d5e6f7a Add user profile pick e6f7g8h more changes pick f7g8h9i Actually fix the bug pick g8h9i0j Add tests pick h9i0j1k Final tweaks

Rebase commands

Change `pick` to one of these:

Command	Shortcut	What it does
pick	p	Keep this commit as is
reword	r	Keep commit but edit the message
edit	e	Stop here to amend the commit
squash	s	Combine with previous commit
fixup	f	Combine with previous, discard this message
drop	d	Delete this commit entirely

Example — cleaning up history

Before:

pick a3f9c2b Add login page pick b8d2f1a wip pick c4d5e6f fix typo pick d5e6f7a Add user profile

After editing in the rebase editor:

pick a3f9c2b Add login page fixup b8d2f1a wip fixup c4d5e6f fix typo reword d5e6f7a Add user profile

Result:

- `wip` and `fix typo` are absorbed into `Add login page`
- You get to rewrite the message for `Add user profile`
- History goes from 4 commits to 2 clean commits

The golden rule of rebase

Never rebase commits that have already been pushed to a shared branch.

Rebase rewrites history — it creates new commits with new SHA hashes. If teammates have already pulled the original commits, their history will diverge from yours and cause serious conflicts.

Use interactive rebase only on LOCAL commits that have not been pushed yet.

8.10 Choosing the Right Command

Situation	Command to use
Need to switch branches but have unfinished work	`git stash`
Bad commit already pushed to shared branch	`git revert`
Bad commits NOT yet pushed, keep changes	`git reset --soft`
Bad commits NOT yet pushed, discard changes	`git reset --hard`
Lost commits after reset	`git reflog`
Need one commit from another branch	`git cherry-pick`
Need to find which commit broke something	`git bisect`

Need to know who wrote a specific line	`git blame`
Want to clean up commits before a PR	`git rebase -i`

Chapter Summary

Command	Key point
`git stash`	Temporarily shelve uncommitted work
`git stash pop`	Restore shelved work
`git revert`	Safe undo — adds a new commit, safe for shared branches
`git reset --soft`	Erase commits, keep changes staged
`git reset --mixed`	Erase commits, keep changes unstaged
`git reset --hard`	Erase commits AND changes permanently
`git reflog`	View full HEAD movement history — ultimate safety net
`git cherry-pick`	Copy a specific commit to current branch
`git bisect`	Binary search to find which commit broke something
`git blame`	Show who last changed each line of a file
`git rebase -i`	Interactively rewrite local commit history

Assessment — Test Yourself

Question 1 You are halfway through writing a new feature when your manager asks you to fix an urgent bug on main immediately. What is the exact sequence of commands?

Question 2 What is the fundamental difference between `git revert` and `git reset`? When must you use revert instead of reset?

Question 3 You ran `git reset --hard HEAD~3` and lost three commits of important work. Is this recoverable? If so, how?

Question 4 You find a bug on line 47 of `app.py`. You want to know who introduced it and when. What command do you run and how do you read the output?

Question 5 Your feature branch has these commits before you open a PR:

Add login feature wip - halfway through fix spacing actually fix spacing test test test final version hopefully

How do you clean this up to one meaningful commit?

Question 6 You need to apply a security fix from your `hotfix` branch to both `main` and `release-v2` without merging the entire hotfix branch into either. What command do you use?

Answers

Answer 1

git stash ← shelve current work git switch main ← move to main git switch -c fix/urgent-bug ← create fix branch [make the fix] git add . git commit -m "Fix urgent bug" git push origin fix/urgent-bug [open PR and merge] git switch feature/my-feature ← return to your feature git stash pop ← restore your work

Answer 2 `git revert` creates a new commit that undoes the changes of a previous commit. History is preserved — the original commit stays. Safe for shared branches because it does not rewrite history.

`git reset` moves HEAD backward to a previous commit, effectively erasing commits from history. It rewrites history — the erased commits disappear from `git log`.

You MUST use `git revert` (not reset) when:

- The commit has already been pushed to a shared branch
- Teammates have already pulled the commit
- You need to preserve the complete audit trail

Answer 3 Yes — recoverable using `git reflog`.

git reflog

Find the SHA of the commit that was HEAD before the reset — it will show as `HEAD@{1}` or similar.

git reset --hard THAT-SHA

Your three commits and all their changes are restored. This works because `git reset --hard` removes commits from `git log` but reflog retains them for 90 days.

Answer 4

git blame app.py

Find line 47 in the output. Read across:

- First column: SHA hash of the commit that last changed this line
- Second column: author name
- Third column: date and time
- Fourth column: line number
- Remaining: the actual code

To see the full context of that commit:

git show SHA-FROM-BLAME

This shows the complete commit — message, author, date, and every line that changed — giving full context for why line 47 was written that way.

Answer 5 Use interactive rebase:

git rebase -i HEAD~6

In the editor change to:

pick Add login feature fixup wip - halfway through fixup fix spacing fixup actually fix spacing fixup test test test fixup final version hopefully

`fixup` absorbs all 5 messy commits into the first one, discarding their messages. Result: one clean commit `Add login feature` appears on the PR — professional and readable.

Answer 6 `git cherry-pick`

First apply to main:

git switch main git cherry-pick HOTFIX-COMMIT-SHA git push

Then apply to release-v2:

git switch release-v2 git cherry-pick HOTFIX-COMMIT-SHA git push

The same fix is now applied to both branches without merging the entire hotfix branch into either.

What is Next

In Chapter 9 we explore GitHub Actions — the automation engine built into GitHub. You will learn to run tests, check code quality and deploy applications automatically every time you push.

Proceed to Chapter 9 — GitHub Actions and CI/CD

Chapter 9 — GitHub Actions and CI/CD

Learning Objectives

By the end of this chapter you will be able to:

- Explain what CI/CD means and why it matters
- Understand the structure of a GitHub Actions workflow file
- Create your first workflow that runs automatically on push
- Understand triggers, jobs and steps
- Use the GitHub Actions marketplace
- Set up automated testing in a workflow
- Deploy a website automatically on every push
- Use secrets to store sensitive information securely
- Read and debug workflow run logs

9.1 What is CI/CD?

CI/CD stands for **Continuous Integration** and **Continuous Deployment** (or Delivery).

Continuous Integration (CI)

Every time a developer pushes code, automated checks run:

- Tests execute to verify nothing is broken
- Code style is checked for consistency
- Security vulnerabilities are scanned
- Build succeeds without errors

If any check fails, the team is notified immediately. The problem is caught within minutes rather than days.

Continuous Deployment (CD)

When code passes all checks and is merged to main, it is automatically deployed to the live server. No manual steps. No human error. Code goes from commit to production automatically.

Without CI/CD: With CI/CD:

Push code Push code | | Hope it works Tests run automatically | | Manually test Checks pass or fail | | Manually deploy Auto-deploy if all pass | | Discover bugs in production Bugs caught before users see them

---

9.2 What is GitHub Actions?

GitHub Actions is GitHub's built-in CI/CD platform. It is free for public repositories and has generous free allowances for private repos.

When you push code, GitHub Actions:

1. Spins up a fresh virtual machine (Ubuntu, Windows or Mac)
2. Clones your repository onto it
3. Runs whatever commands you define
4. Reports success or failure

5. Optionally deploys your application

The entire process is defined in a YAML file that lives inside your repository.

9.3 The Workflow File

Everything in GitHub Actions is controlled by a workflow file.

Location

your-repo/ .github/ workflows/ main.yml ← your workflow file lives here

The file must be inside `.github/workflows/`. The filename can be anything ending in `.yml` or `.yaml`.

Basic structure

```yaml
name: My First Workflow

on:
  push:
    branches: [ main ]

jobs:
  my-job:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Say hello
        run: echo "Hello from GitHub Actions!"
```

9.4 Understanding the Structure

name

```yaml
name: My First Workflow
```

The display name shown on the Actions tab on GitHub. Can be anything descriptive.

on — triggers

```yaml
on:
  push:
    branches: [ main ]
```

Defines what events start the workflow. Common triggers:

Trigger	When it fires
`push`	Every time commits are pushed
`pull_request`	When a PR is opened or updated
`schedule`	On a cron schedule (e.g. daily at midnight)
`workflow_dispatch`	Manually triggered from GitHub UI
`release`	When a new release is published

Multiple triggers:

```yaml
on:
  push:
    branches: [ main ]
  pull_request:
    branches: [ main ]
  schedule:
    - cron: '0 9 * * 1'  # Every Monday at 9am
```

jobs

```yaml
jobs:
  my-job:
    runs-on: ubuntu-latest
```

A workflow can have multiple jobs. Each job:

- Runs on its own fresh virtual machine
- Runs in parallel with other jobs by default
- Can be configured to depend on other jobs

Available runners:

Runner	Operating system
`ubuntu-latest`	Latest Ubuntu Linux
`windows-latest`	Latest Windows Server
`macos-latest`	Latest macOS

steps

```yaml
    steps:
      - name: Checkout code
        uses: actions/checkout@v4
```

```
      - name: Run a command
        run: echo "Hello!"
```

Steps run sequentially within a job. Two types:

uses — runs a pre-built action from the marketplace:

```
uses: actions/checkout@v4
```

run — runs a shell command directly:

```
run: echo "Hello from GitHub Actions!"
```

9.5 Your First Workflow

Let us create a real workflow step by step.

Create the workflow file

In your terminal:

mkdir -p .github/workflows

Create the file:

code .github/workflows/main.yml

Paste this content:

```
name: My First Workflow

on:
  push:
    branches: [ main ]
  pull_request:
    branches: [ main ]

jobs:
  check-repo:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: List all files
        run: ls -la

      - name: Show Git log
        run: git log --oneline -5
```

```
      - name: Say hello
        run: echo "Hello from GitHub Actions!"
```

Commit and push

git add .github/ git commit -m "Add first GitHub Actions workflow" git push

View the workflow run

Go to your GitHub repo → click **Actions** tab.

You will see your workflow running. Click on it to see each step's output.

9.6 Actions Marketplace

The `uses` keyword references pre-built actions from the GitHub Actions Marketplace — thousands of ready-made building blocks contributed by GitHub and the community.

Common marketplace actions

Action	What it does
`actions/checkout@v4`	Downloads your repo onto the runner
`actions/setup-node@v4`	Installs Node.js
`actions/setup-python@v5`	Installs Python
`actions/upload-artifact@v4`	Saves files from the workflow
`actions/cache@v4`	Caches dependencies to speed up runs

Using an action

```yaml
steps:
  - name: Set up Node.js
    uses: actions/setup-node@v4
    with:
      node-version: '20'
```

The `with` keyword passes configuration to the action.

Finding actions

Browse at:

https://github.com/marketplace?type=actions

9.7 Running Tests Automatically

The most common CI use case — run your test suite on every push and pull request.

Node.js example

```yaml
name: Node.js CI

on:
  push:
    branches: [ main ]
  pull_request:
    branches: [ main ]

jobs:
  test:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Set up Node.js
        uses: actions/setup-node@v4
        with:
          node-version: '20'

      - name: Install dependencies
        run: npm install

      - name: Run tests
        run: npm test
```

Python example

```yaml
name: Python CI

on: [push, pull_request]

jobs:
  test:
    runs-on: ubuntu-latest

    steps:
      - uses: actions/checkout@v4

      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: '3.12'

      - name: Install dependencies
        run: pip install -r requirements.txt
```

```
      - name: Run tests
        run: python -m pytest
```

What happens when tests fail

If any step exits with an error, the workflow fails. GitHub shows a red cross on the commit and the PR. Merging can be blocked until tests pass.

9.8 Deploying GitHub Pages Automatically

You can deploy your website to GitHub Pages automatically every time you push to main.

```yaml
name: Deploy to GitHub Pages

on:
  push:
    branches: [ main ]

permissions:
  contents: read
  pages: write
  id-token: write

jobs:
  deploy:
    runs-on: ubuntu-latest
    environment:
      name: github-pages

    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Setup Pages
        uses: actions/configure-pages@v4

      - name: Upload site
        uses: actions/upload-pages-artifact@v3
        with:
          path: '.'

      - name: Deploy to GitHub Pages
        uses: actions/deploy-pages@v4
```

After pushing this workflow, every push to main automatically rebuilds and deploys your site. No manual steps needed ever again.

9.9 Using Secrets

Never put passwords, API keys or tokens directly in your workflow files. They would be visible in your repo.

Instead, use GitHub Secrets — encrypted values stored securely on GitHub.

Creating a secret

GitHub repo → Settings → Secrets and variables → Actions → New repository secret

Add a secret called `MY_API_KEY` with your actual value.

Using a secret in a workflow

```yaml
steps:
  - name: Deploy with API key
    run: deploy-script.sh
    env:
      API_KEY: ${{ secrets.MY_API_KEY }}
```

The `${{ secrets.MY_API_KEY }}` syntax references your secret. The actual value is never visible in logs — GitHub replaces it with `***`.

Common secrets to store

Secret name	What it stores
`DEPLOY_KEY`	SSH key for deployment
`DATABASE_URL`	Database connection string
`API_TOKEN`	Third-party service token
`SLACK_WEBHOOK`	Slack notification URL

9.10 Multi-Job Workflows

Jobs run in parallel by default. Use `needs` to create dependencies between jobs.

```yaml
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: npm test

  build:
    needs: test          ← only runs if test passes
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: npm run build

  deploy:
    needs: build         ← only runs if build passes
    runs-on: ubuntu-latest
```

```
    steps:
      - uses: actions/checkout@v4
      - run: npm run deploy
```

This creates a pipeline:

test --> build --> deploy

If tests fail, build never runs. If build fails, deploy never runs.

9.11 Environment Variables

Pass configuration to your workflow steps.

Workflow-level variables

```
env:
  NODE_ENV: production
  APP_VERSION: 1.0.0

jobs:
  build:
    steps:
      - run: echo "Building version $APP_VERSION"
```

Step-level variables

```
steps:
  - name: Run with config
    run: echo "Environment is $NODE_ENV"
    env:
      NODE_ENV: staging
```

Built-in variables

GitHub provides many variables automatically:

Variable	Value
`${{ github.repository }}`	owner/repo-name
`${{ github.sha }}`	Full commit SHA
`${{ github.ref }}`	Branch or tag ref
`${{ github.actor }}`	Username who triggered the run
`${{ github.event_name }}`	What triggered the workflow

9.12 Reading Workflow Logs

When a workflow runs, every step produces output visible in the Actions tab.

Navigation

GitHub → Actions → [workflow run] → [job name] → [step name]

What to look for when debugging

Symptom	Where to look
Workflow did not start	Check the trigger in `on:`
Step failed	Expand the failed step — read the error message
Wrong Node/Python version	Check the `setup-node` or `setup-python` step
Missing dependency	Check the install step output
Secret not working	Check secret name matches exactly

Re-running a workflow

If a workflow failed due to a flaky test or network error:

Actions → [failed run] → Re-run all jobs

9.13 Status Badges

Add a badge to your README showing the current build status:

```
![CI](https://github.com/USERNAME/REPO/actions/workflows/main.yml/badge.svg)
```

This shows a green "passing" or red "failing" badge in your README — immediately visible to anyone visiting your repo.

Chapter Summary

Concept	Key point
CI/CD	Automated testing and deployment on every push
Workflow file	YAML file in `.github/workflows/`
Trigger	What event starts the workflow
Job	A set of steps running on one virtual machine
Step	A single command or marketplace action
`uses`	Run a pre-built marketplace action

`run`	Execute a shell command
Secrets	Encrypted values for sensitive data
`needs`	Create dependencies between jobs

Assessment — Test Yourself

Question 1 What is the difference between Continuous Integration and Continuous Deployment?

Question 2 Your workflow file is at `.github/workflows/ci.yml`. The workflow is not running when you push. What are three possible causes?

Question 3 Why should you never put an API key directly in a workflow file? What should you do instead?

Question 4 You have three jobs: `test`, `build` and `deploy`. You want `build` to run only if `test` passes, and `deploy` to run only if `build` passes. Write the `needs` configuration for each job.

Question 5 What does `actions/checkout@v4` do and why is it almost always the first step in every workflow?

Question 6 Your tests pass locally but fail in GitHub Actions. Name four things you would check to diagnose the problem.

Answers

Answer 1 Continuous Integration (CI) means automatically running tests and checks every time code is pushed. The goal is to catch bugs and integration problems immediately — within minutes of the push — rather than discovering them days later or in production.

Continuous Deployment (CD) means automatically deploying code to the live server whenever it passes all CI checks and is merged to the main branch. No manual deployment steps. Code goes from a merged PR to production automatically.

Together CI/CD creates a pipeline where code is continuously integrated, tested and delivered to users.

Answer 2 Three possible causes:

1. Wrong trigger — the `on:` section does not include `push` or specifies the wrong branch name
2. Wrong branch — the workflow runs on `master` but you are pushing to `main` (or vice versa)
3. Syntax error — the YAML file has an indentation or formatting error that prevents GitHub from parsing it. Check the Actions tab for a workflow file parse error.

Answer 3 If you put an API key directly in a workflow file, it is visible to anyone who can view your repository — including the public if the repo is public. Even in private repos, everyone with access can see it. If the repo becomes public or is compromised, the key is exposed permanently in the Git history.

Instead, store sensitive values as GitHub Secrets: Repository → Settings → Secrets and variables → Actions Reference in the workflow as `${{ secrets.SECRET_NAME }}` The value is encrypted and never visible in logs.

Answer 4

```yaml
jobs:
  test:
    runs-on: ubuntu-latest
```

```
    steps:
      - run: npm test

  build:
    needs: test
    runs-on: ubuntu-latest
    steps:
      - run: npm run build

  deploy:
    needs: build
    runs-on: ubuntu-latest
    steps:
      - run: npm run deploy
```

Answer 5 `actions/checkout@v4` downloads your repository onto the fresh virtual machine that GitHub Actions has spun up. The machine starts completely empty — it has no knowledge of your code. Without this step, all subsequent steps would fail because there are no files to work with. It is almost always the first step because everything else — running tests, building the app, checking code style — requires access to your source code.

Answer 6 Four things to check:

1. Node.js / Python version — the version in Actions may differ from your local version. Specify the exact version in `setup-node` or `setup-python`.
2. Environment variables — your local machine may have environment variables set that the Actions runner does not. Check if any tests depend on env vars.
3. Operating system differences — you may be on Windows locally but the runner is Ubuntu. File paths, line endings and commands differ.
4. Missing dependencies — your local machine may have globally installed packages that are not installed in the workflow. Check the install step and ensure all dependencies are in your requirements file.

What is Next

In Chapter 10 we put everything together in a real project. You will build and deploy a professional website using Git, GitHub, GitHub Pages and GitHub Actions — applying every skill from this book in one complete project.

Proceed to Chapter 10 — Real Project — Build and Deploy a Website

Chapter 10 — Real Project — Build and Deploy a Website

Learning Objectives

By the end of this chapter you will be able to:

- Apply every skill from this book in one complete project
- Build a professional website from scratch using HTML and CSS
- Host it live on the internet for free using GitHub Pages
- Set up automatic deployment using GitHub Actions
- Update the live site by simply pushing to GitHub
- Structure a multi-page website with navigation
- Use branches and pull requests for website changes
- Add a custom domain to your GitHub Pages site

10.1 Project Overview

In this chapter you build a real professional website and deploy it live using everything you have learned.

What you will build

A personal professional website with:

- A landing page with your name and introduction
- An about page with your background
- A projects page linking to your GitHub repos
- A contact page
- Navigation between all pages
- Automatic deployment on every push

What you will use

Technology	Purpose
HTML5	Page structure and content
CSS3	Styling and layout
Git	Version control
GitHub	Remote repository
GitHub Pages	Free website hosting
GitHub Actions	Automatic deployment

The end result

Your website will be live at:

https://YOUR-USERNAME.github.io

Accessible to anyone in the world, for free, forever.

10.2 Understanding GitHub Pages

GitHub Pages is a free static website hosting service built into GitHub. It serves HTML, CSS and JavaScript files directly from a repository.

How it works

You push HTML files to GitHub | GitHub Pages detects the push | Files are served at your-username.github.io | Anyone in the world can visit your site

Two types of GitHub Pages sites

Type	URL	Repository name
User site	username.github.io	Must be named `username.github.io`
Project site	username.github.io/project	Any repo name

For a personal portfolio you want the user site — your name as the URL.

10.3 Setting Up the Repository

Step 1 — Create the repository on GitHub

Go to `https://github.com/new`

Repository name: `YOUR-USERNAME.github.io`

Replace `YOUR-USERNAME` with your actual GitHub username. This exact naming is required for a user GitHub Pages site.

Settings:

- Public
- No README, no .gitignore, no licence

Click **Create repository**.

Step 2 — Clone locally

cd ~/Desktop git clone https://github.com/YOUR-USERNAME/YOUR-USERNAME.github.io.git cd YOUR-USERNAME.github.io

Or if you prefer to initialise locally:

mkdir YOUR-USERNAME.github.io cd YOUR-USERNAME.github.io git init git remote add origin https://github.com/YOUR-USERNAME/YOUR-USERNAME.github.io.git

---

10.4 Project Structure

Before writing any code, plan the structure:

YOUR-USERNAME.github.io/ index.html ← landing page (required) about.html ← about page projects.html ← projects page contact.html ← contact page css/ style.css ← all styling images/ profile.jpg ← your photo .github/ workflows/ deploy.yml ← auto-deploy workflow

`index.html` is the required entry point. When someone visits your URL, GitHub Pages serves `index.html`.

10.5 Building the Landing Page

Create `index.html`:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Your Name — Professional Title</title>
  <link rel="stylesheet" href="css/style.css">
</head>
<body>

  <!-- Navigation -->
  <nav>
    <div class="logo">Your Name</div>
    <ul>
      <li><a href="index.html">Home</a></li>
      <li><a href="about.html">About</a></li>
      <li><a href="projects.html">Projects</a></li>
      <li><a href="contact.html">Contact</a></li>
    </ul>
  </nav>

  <!-- Hero section -->
  <section class="hero">
    <h1>Your Full Name</h1>
    <h2>Your Professional Title</h2>
    <p>
      A short introduction about yourself — what you do,
      what you are passionate about, and what makes you
      unique. Keep it to 2-3 sentences.
    </p>
    <a href="about.html" class="btn">Learn More</a>
    <a href="contact.html" class="btn-outline">Get in Touch</a>
  </section>

  <!-- Quick links -->
  <section class="quick-links">
    <h3>What I Do</h3>
    <div class="cards">
      <div class="card">
        <h4>Skill or Service 1</h4>
        <p>Brief description of this skill or service.</p>
```

```
      </div>
      <div class="card">
        <h4>Skill or Service 2</h4>
        <p>Brief description of this skill or service.</p>
      </div>
      <div class="card">
        <h4>Skill or Service 3</h4>
        <p>Brief description of this skill or service.</p>
      </div>
    </div>
  </section>

  <!-- Footer -->
  <footer>
    <p>Your Name — Your Title</p>
    <p>
      <a href="https://github.com/YOUR-USERNAME">GitHub</a>
    </p>
  </footer>

</body>
</html>
```

10.6 Adding Styles

Create the `css` folder and `style.css`:

```
/* Reset */
* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
}

/* Base */
body {
  font-family: Georgia, serif;
  background: #FAFAFA;
  color: #333;
  line-height: 1.6;
}

/* Navigation */
nav {
  background: #2C3E50;
  padding: 16px 40px;
  display: flex;
  justify-content: space-between;
  align-items: center;
}
```

```css
nav .logo {
  color: white;
  font-size: 20px;
  font-weight: bold;
}

nav ul {
  list-style: none;
  display: flex;
  gap: 28px;
}

nav ul li a {
  color: white;
  text-decoration: none;
  font-size: 15px;
  transition: opacity 0.2s;
}

nav ul li a:hover {
  opacity: 0.75;
}

/* Hero */
.hero {
  background: linear-gradient(135deg, #2C3E50, #3498DB);
  color: white;
  padding: 100px 40px;
  text-align: center;
}

.hero h1 {
  font-size: 48px;
  margin-bottom: 12px;
}

.hero h2 {
  font-size: 24px;
  font-weight: normal;
  opacity: 0.85;
  margin-bottom: 20px;
}

.hero p {
  font-size: 18px;
  max-width: 600px;
  margin: 0 auto 36px;
  opacity: 0.9;
}

/* Buttons */
```

```css
.btn {
  background: white;
  color: #2C3E50;
  padding: 14px 32px;
  border-radius: 30px;
  text-decoration: none;
  font-size: 16px;
  margin: 0 8px;
  display: inline-block;
  font-weight: bold;
}

.btn-outline {
  background: transparent;
  color: white;
  border: 2px solid white;
  padding: 12px 32px;
  border-radius: 30px;
  text-decoration: none;
  font-size: 16px;
  margin: 0 8px;
  display: inline-block;
}

/* Quick links */
.quick-links {
  padding: 70px 40px;
  text-align: center;
}

.quick-links h3 {
  font-size: 32px;
  margin-bottom: 40px;
  color: #2C3E50;
}

.cards {
  display: flex;
  flex-wrap: wrap;
  gap: 24px;
  justify-content: center;
}

.card {
  background: white;
  border-radius: 12px;
  padding: 32px 28px;
  width: 260px;
  box-shadow: 0 2px 12px rgba(0,0,0,0.08);
  transition: transform 0.2s;
}
```

```css
.card:hover {
  transform: translateY(-4px);
}

.card h4 {
  font-size: 18px;
  color: #2C3E50;
  margin-bottom: 10px;
}

.card p {
  font-size: 14px;
  color: #666;
}

/* Footer */
footer {
  background: #2C3E50;
  color: #ccc;
  text-align: center;
  padding: 28px;
  font-size: 14px;
}

footer a {
  color: #3498DB;
  text-decoration: none;
}

/* Responsive */
@media (max-width: 768px) {
  nav {
    flex-direction: column;
    gap: 16px;
    padding: 16px;
  }

  nav ul {
    gap: 16px;
    flex-wrap: wrap;
    justify-content: center;
  }

  .hero {
    padding: 60px 20px;
  }

  .hero h1 {
    font-size: 32px;
  }

  .quick-links {
```

```
    padding: 40px 20px;
  }
}
```

10.7 Your First Commit

git add . git commit -m "Add landing page and styles" git push -u origin main

Visit `https://YOUR-USERNAME.github.io` in your browser.

GitHub Pages may take 1 to 2 minutes to deploy the first time. Refresh after a couple of minutes and your site will be live.

10.8 Adding More Pages

Each additional page follows the same pattern. Copy the navigation and footer from `index.html` and add your content in between.

about.html structure

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>About — Your Name</title>
  <link rel="stylesheet" href="css/style.css">
</head>
<body>

  <nav>
    <!-- Same navigation as index.html -->
  </nav>

  <section class="page-hero">
    <h1>About Me</h1>
  </section>

  <section class="content">
    <h2>My Story</h2>
    <p>Write your professional background here.</p>

    <h2>Skills</h2>
    <ul>
      <li>Skill 1</li>
      <li>Skill 2</li>
      <li>Skill 3</li>
    </ul>

    <h2>Experience</h2>
```

```
    <p>Your career history here.</p>
  </section>

  <footer>
    <!-- Same footer as index.html -->
  </footer>

</body>
</html>
```

Add to `style.css`:

```
.page-hero {
  background: #2C3E50;
  color: white;
  padding: 60px 40px;
  text-align: center;
}

.page-hero h1 {
  font-size: 40px;
}

.content {
  max-width: 800px;
  margin: 60px auto;
  padding: 0 40px;
}

.content h2 {
  font-size: 24px;
  color: #2C3E50;
  margin: 32px 0 12px;
}

.content p {
  font-size: 16px;
  color: #555;
  line-height: 1.8;
  margin-bottom: 16px;
}

.content ul {
  padding-left: 20px;
  color: #555;
  line-height: 2;
}
```

10.9 Using Branches for Changes

Every change to your live website should go through a branch. This protects your site from broken code reaching production.

Workflow for adding a new page

Step 1: Create a branch git switch -c add-projects-page

Step 2: Create projects.html [write the page]

Step 3: Commit git add projects.html git commit -m "Add projects page"

Step 4: Push the branch git push origin add-projects-page

Step 5: Open a pull request on GitHub

Step 6: Review the changes

Step 7: Merge the PR

Step 8: Delete the branch git switch main git pull git branch -d add-projects-page

Why this matters for a website

If you push broken HTML directly to main, your live site breaks immediately and visitors see an error.

With branches and pull requests:

- You can preview the change before it goes live
- You can easily revert if something breaks
- Your history shows exactly when and why each change was made

---

10.10 Setting Up Auto-Deploy

Create `.github/workflows/deploy.yml` :

```
name: Deploy to GitHub Pages

on:
  push:
    branches: [ main ]

permissions:
  contents: read
  pages: write
  id-token: write

concurrency:
  group: pages
  cancel-in-progress: true

jobs:
  deploy:
    runs-on: ubuntu-latest
    environment:
      name: github-pages
```

```
      url: ${{ steps.deployment.outputs.page_url }}

    steps:
      - name: Checkout
        uses: actions/checkout@v4

      - name: Setup Pages
        uses: actions/configure-pages@v4

      - name: Upload artifact
        uses: actions/upload-pages-artifact@v3
        with:
          path: '.'

      - name: Deploy to GitHub Pages
        id: deployment
        uses: actions/deploy-pages@v4
```

Enable GitHub Pages in your repository settings:

Repository → Settings → Pages → Source → GitHub Actions

Now every push to main automatically rebuilds and deploys your site within about 60 seconds.

10.11 Adding Your Profile Photo

Create an `images` folder and add your photo:

mkdir images

Copy your photo into the images folder then reference it in your HTML:

```
<img src="images/profile.jpg" alt="Your Name"
     style="width: 150px; height: 150px;
            border-radius: 50%; object-fit: cover;">
```

Or in CSS:

```
.profile-photo {
  width: 150px;
  height: 150px;
  border-radius: 50%;
  object-fit: cover;
  border: 4px solid white;
  box-shadow: 0 4px 16px rgba(0,0,0,0.2);
}
```

10.12 Adding a Custom Domain

If you own a domain name (e.g. `yourname.com`), you can point it to your GitHub Pages site for free.

Step 1 — Add your domain on GitHub

Repository → Settings → Pages → Custom domain

Enter your domain: `yourname.com`

Click Save. GitHub creates a CNAME file in your repo.

Step 2 — Configure your DNS

At your domain registrar (GoDaddy, Namecheap, etc.), add these DNS records:

For an apex domain (yourname.com):

Type: A Name: @ Values: 185.199.108.153 185.199.109.153 185.199.110.153 185.199.111.153

For www subdomain:

Type: CNAME Name: www Value: YOUR-USERNAME.github.io

DNS changes take up to 48 hours to propagate.

Step 3 — Enable HTTPS

Once the domain is verified:

Repository → Settings → Pages → Enforce HTTPS ✓

Your site is now served securely at `https://yourname.com`.

10.13 The Complete Project Workflow

Once your site is set up, maintaining it is simple:

Adding a new article or page

git switch -c add-new-article [create the new file] git add . git commit -m "Add article: [title]" git push origin add-new-article [open PR on GitHub] [merge PR] git switch main git pull

Site updates automatically within 60 seconds of merging.

Fixing a typo

git switch -c fix/typo-homepage [fix the typo] git add index.html git commit -m "Fix typo in homepage hero text" git push origin fix/typo-homepage [open and merge PR]

Emergency fix

git switch -c hotfix/broken-navigation [fix the issue] git add . git commit -m "Fix broken navigation links" git push origin hotfix/broken-navigation [open and immediately merge PR]

Chapter Summary

Topic	Key point
GitHub Pages	Free hosting for static HTML/CSS/JS sites

User site	Repo named `username.github.io` — served at that URL
index.html	Required entry point — GitHub Pages serves this first
Auto-deploy	Push to main → Actions runs → site updates in 60s
Branch workflow	Always use branches for changes to live sites
Custom domain	Point any domain to your GitHub Pages site for free
HTTPS	GitHub provides free SSL certificates automatically

Assessment — Test Yourself

Question 1 What is the required repository name for a personal GitHub Pages user site? Why must it follow this format?

Question 2 You push changes to your GitHub Pages site but the live site has not updated after 5 minutes. What do you check?

Question 3 Why should you use branches even for a personal website that only you maintain?

Question 4 Your navigation bar HTML is identical on all four pages of your website. Every time you add a new page you must update four files. What is the professional solution to this problem?

Question 5 You want your website to be accessible at both `yourname.com` and `www.yourname.com`. What DNS records do you need to configure?

Answers

Answer 1 The repository must be named exactly `USERNAME.github.io` where USERNAME matches your GitHub username exactly, including capitalisation. GitHub uses this special naming convention to identify which repository should be served as the user's personal GitHub Pages site. Any other name would create a project site at `username.github.io/repo-name` instead of the root URL `username.github.io`.

Answer 2 Check in this order:

1. Actions tab — is the deployment workflow running? Did it pass or fail?
2. GitHub Pages settings — is Pages enabled and set to the correct source (main branch or GitHub Actions)?
3. Browser cache — hard refresh with Ctrl+Shift+R to bypass cached content
4. File names — is `index.html` correctly named? GitHub Pages is case-sensitive on some systems

Answer 3 Even for a solo personal site, branches provide:

- Safety — broken code on a branch does not break the live site
- Review opportunity — you can compare the branch against main before merging, catching mistakes before visitors see them
- History — the git log shows exactly what changed and when, making it easy to revert to a previous state
- Good habits — practising branch workflow on personal projects prepares you for team environments

Answer 4 The professional solution is to use a static site generator or templating system — for example Jekyll (supported natively by GitHub Pages), Hugo or Eleventy. These tools let you define the navigation once in a template and render it across all pages automatically. Alternatively, for a simple site, JavaScript can inject shared components. For a pure HTML site, accept the duplication but be disciplined about updating all pages together.

Answer 5 For the apex domain `yourname.com`:

Type: A Name: @ Values: 185.199.108.153, 185.199.109.153, 185.199.110.153, 185.199.111.153

For the www subdomain:

Type: CNAME Name: www Value: YOUR-USERNAME.github.io

Then in GitHub Pages settings, set your custom domain to `yourname.com`. GitHub will automatically redirect `www.yourname.com` to `yourname.com`. Enable HTTPS enforcement after the domain is verified.

What is Next

In the final chapter we step back and look at the bigger picture — professional Git practices, team conventions, career advice and how to continue learning.

Proceed to Chapter 11 — Best Practices

Chapter 11 — Best Practices

Learning Objectives

By the end of this chapter you will be able to:

- Write commit messages that communicate clearly
- Choose the right branching strategy for your team
- Structure repositories professionally
- Use Git aliases to work faster
- Understand common team conventions
- Avoid the most costly Git mistakes
- Build a GitHub profile that impresses employers
- Plan your next steps for continued learning

11.1 The Mark of a Professional Git User

There is a significant difference between someone who knows Git commands and someone who uses Git professionally.

The commands are the easy part. The hard part is judgment — knowing when to commit, what to write, how to structure your history, and how to work with others without causing problems.

This chapter is about developing that judgment.

11.2 Writing Great Commit Messages

The commit message is a letter to your future self and your teammates. It will be read months or years from now when someone is trying to understand why a decision was made.

The golden rule

A commit message should complete this sentence:

> *"If applied, this commit will..."*

The format

Short summary (under 72 characters)

Optional longer description if needed. Explain WHY this change was made, not WHAT changed (the diff shows what changed).

Include any relevant context:

Issue number being fixed Why this approach was chosen What alternatives were considered Any side effects or limitations

Good vs bad commit messages

Bad	Good

fix	Fix null pointer error in login validation
update	Update user dashboard to show recent activity
wip	Add draft of payment processing — not yet tested
changes	Remove deprecated getUserData function
stuff	Refactor database queries to use connection pooling

The seven rules of great commit messages

1. Separate subject from body with a blank line
2. Limit the subject line to 72 characters
3. Capitalise the subject line
4. Do not end the subject line with a period
5. Use the imperative mood ("Add" not "Added" or "Adding")
6. Wrap the body at 72 characters
7. Use the body to explain what and why, not how

Conventional Commits

Many teams use the Conventional Commits specification — a structured format for commit messages:

type(scope): description

feat(auth): add OAuth2 login with Google fix(nav): correct mobile menu alignment docs(readme): update installation instructions refactor(db): simplify query builder test(auth): add unit tests for login flow chore(deps): update dependencies to latest

Types:

- `feat` — new feature
- `fix` — bug fix
- `docs` — documentation only
- `style` — formatting, no code change
- `refactor` — code restructuring
- `test` — adding tests
- `chore` — maintenance tasks

11.3 Branching Strategies

GitHub Flow (recommended for most teams)

main is always deployable Every change goes through a branch and PR Simple, effective, works at any scale

main | +---> feature/user-auth -----> PR --> main | +---> fix/login-bug ----------> PR --> main | +---> docs/update-readme -----> PR --> main

Best for: Web applications, continuous deployment, teams that deploy frequently.

GitFlow (for versioned software)

main — production releases only develop — integration branch feature/* — new features release/* — release preparation hotfix/* — emergency production fixes

Best for: Mobile apps, libraries, versioned software with scheduled releases.

Trunk-Based Development

Everyone commits directly to main (or short-lived branches) Feature flags hide incomplete features in production Requires strong automated testing

Best for: Large experienced teams with strong CI/CD.

Which to choose

Team size	Deployment frequency	Recommended strategy
Solo	Whenever ready	GitHub Flow
Small (2-5)	Continuous	GitHub Flow
Medium (5-20)	Continuous	GitHub Flow
Large (20+)	Scheduled releases	GitFlow
Large (20+)	Continuous	Trunk-based

11.4 Repository Structure Best Practices

The essential files

Every professional repository should have:

repo/ README.md ← required — explains the project .gitignore ← required — excludes unwanted files LICENSE ← important for open source CONTRIBUTING.md ← optional — how to contribute CHANGELOG.md ← optional — history of changes

Writing a great README

A README should answer five questions:

1. **What is this?** — One sentence description
2. **Why should I care?** — The problem it solves
3. **How do I install it?** — Step-by-step setup
4. **How do I use it?** — Basic usage examples
5. **How do I contribute?** — For open source projects

README template

```
# Project Name

One sentence describing what this project does.

## Why

The problem this project solves and why it matters.

## Installation
```

```
Step by step instructions to get this running.

## Usage

Basic usage examples with code snippets.

## Contributing

How others can contribute to this project.

## Licence

MIT Licence — see LICENSE file for details.
```

11.5 .gitignore Best Practices

What to always ignore

Dependencies — regeneratable, often huge

node_modules/ vendor/ venv/

Build output — regeneratable

dist/ build/ *.pyc pycache/

Environment and secrets — NEVER commit these

.env .env.local .env.production *.pem *.key config/secrets.yml

Editor files — personal preference, not project files

.vscode/ .idea/ *.swp .DS_Store Thumbs.db

Logs — generated at runtime

*.log logs/

Test coverage — generated

coverage/ .coverage

Using gitignore.io

Generate a perfect `.gitignore` for your stack:

https://www.gitignore.io

Enter your languages and tools — it generates the complete ignore file automatically.

If you accidentally committed a secret

1. Rotate the secret immediately — assume it is compromised
2. Remove from tracking: `git rm --cached secret-file`
3. Commit the removal

4. Rewrite history to remove it from all past commits (use `git filter-branch` or BFG Repo Cleaner)
5. Force push the rewritten history
6. Notify your team

11.6 Git Aliases — Work Faster

Aliases save time on commands you type dozens of times per day. Set them once and use them forever.

Essential aliases

git config --global alias.st status git config --global alias.co checkout git config --global alias.br branch git config --global alias.ci commit git config --global alias.lg "log --oneline --graph --all" git config --global alias.last "log -1 HEAD" git config --global alias.unstage "restore --staged" git config --global alias.undo "reset HEAD~1 --mixed"

Using aliases

git st instead of git status git lg instead of git log --oneline --graph --all git last instead of git log -1 HEAD git unstage instead of git restore --staged git undo instead of git reset HEAD~1 --mixed

Viewing all aliases

git config --global --list | grep alias

11.7 The Most Costly Git Mistakes

Learn from others' mistakes so you do not make them yourself.

Mistake 1 — Committing secrets

What happens: API keys, passwords or tokens committed to a public repo are scraped by bots within minutes. The credentials are compromised even after you delete them because the history remains.

Prevention:

- Always add secrets to `.gitignore` before creating them
- Use environment variables, not hardcoded values
- Use tools like `git-secrets` to scan commits automatically

Mistake 2 — Force pushing to shared branches

What happens: `git push --force` rewrites history on the remote. Everyone who has pulled those commits now has a diverged history. Their next push or pull fails.

Prevention:

- Never force push to `main` or any shared branch
- Use `git push --force-with-lease` instead — it fails if someone else has pushed since your last fetch
- Protect branches in GitHub settings

Mistake 3 — Huge commits

What happens: One massive commit with 50 files changed is impossible to review, impossible to revert cleanly, and impossible to understand months later.

Prevention:

- Commit early and often
- Each commit should do one thing
- If you struggle to write a message, the commit is too big

Mistake 4 — Committing to the wrong branch

What happens: You intended to work on `feature/x` but forgot to switch — your commits went to `main`.

Prevention:

- Always run `git status` before committing to verify which branch you are on
- Fix: use `git reset` to move commits to the right branch

Mistake 5 — Not pulling before pushing

What happens: You push and get rejected because the remote has new commits. You pull, get merge conflicts, panic and make things worse.

Prevention:

- Always `git pull` before starting work each day
- Pull before pushing at the end of the day

Mistake 6 — Ignoring merge conflicts

What happens: During a conflict, the developer accepts all changes without reading them. Broken code or duplicated content reaches main.

Prevention:

- Read every conflict carefully
- Understand both versions before choosing
- Test the code after resolving

11.8 Code Review Best Practices

As the author

- Keep pull requests small and focused
- Write a clear description explaining the why not the what
- Link to the relevant issue
- Add screenshots for UI changes
- Respond to all comments — even to say "good point, fixed"
- Do not take feedback personally — the code is being reviewed, not you

As the reviewer

- Be specific — quote the line you are commenting on
- Be constructive — suggest improvements, not just criticism
- Distinguish blocking vs non-blocking comments
- Approve when satisfied — do not leave PRs in limbo
- Review promptly — do not let PRs wait more than 24 hours
- Praise good work — acknowledge clever solutions

Comment tone

Instead of	Say
"This is wrong"	"This could cause X issue when Y happens"
"Why did you do this?"	"Could you help me understand the approach here?"
"Fix this"	"Consider using X pattern here — it handles the edge case of Y"
"Bad code"	"I think we could simplify this by..."

11.9 Building a GitHub Profile That Impresses

Your GitHub profile is your professional portfolio. It works for you 24 hours a day.

Profile essentials

- **Professional photo** — clear, well-lit headshot
- **Real name** — not a username
- **Bio** — one sentence: what you do and what you are passionate about
- **Website** — link to your portfolio or professional site
- **Location** — optional but builds trust
- **Company** — current employer or organisation

Pinned repositories

Pin your best 6 repositories:

Profile → Customize your pins

Choose repos that show:

- Range of skills
- Real completed projects
- Good README files
- Recent activity

The contribution graph

The green squares on your profile show your daily Git activity. A consistent contribution graph signals an active, engaged developer.

How to keep it green:

- Commit something every day — even documentation
- Work on personal projects
- Contribute to open source
- Write blog posts in a repo

The profile README

Create a repo named exactly `YOUR-USERNAME` and add a `README.md`. It appears at the top of your profile.

Use it to:

- Introduce yourself
- List your skills and tools

- Show your stats
- Link to your best work

Example profile README:

```
# Hi, I am Dr M Quamrul Hassan

Senior Consultant Paediatrician & Neonatologist
Evercare Hospital Dhaka, Bangladesh

## What I do
- Paediatric and neonatal medicine since 1992
- Research in infectious diseases and child nutrition
- Medical education and teaching
- Building tools to improve patient care

## My work
- [Professional website](https://mqhassan.github.io)
- [Research portfolio](https://github.com/MQHassan/research-portfolio)
- [CV and Biodata](https://github.com/MQHassan/cv-biodata)

## Currently learning
- Python for medical data analysis
- GitHub Actions for automation
```

11.10 Your Next Steps

You have completed this book. Here is how to continue growing.

Immediate actions (this week)

[ ] Create your GitHub profile README [ ] Pin your best 6 repositories [ ] Complete your GitHub profile [ ] Push something every day this week [ ] Open your first pull request on an open source project

Short term (next month)

[ ] Learn the basics of one programming language (Python is recommended for its versatility) [ ] Build one complete project and deploy it [ ] Contribute to one open source project [ ] Read other people's code on GitHub [ ] Set up CI/CD on one of your projects

Medium term (next 6 months)

[ ] Build 3-5 portfolio projects [ ] Learn SQL for data management [ ] Understand basic web development (HTML, CSS, JavaScript) [ ] Participate in Hacktoberfest (October) [ ] Start a technical blog using GitHub Pages

Resources for continued learning

Resource	What you will learn
`https://learngitbranching.js.org`	Interactive Git branching visualiser
`https://ohshitgit.com`	How to fix common Git disasters

`https://www.atlassian.com/git`	Comprehensive Git tutorials
`https://docs.github.com`	Official GitHub documentation
`https://github.com/EbookFoundation/free-programming-books`	Free programming books
`https://exercism.org`	Practice programming with mentorship

11.11 The Complete Git Command Reference

Everything you have learned in one place.

Setup

git config --global user.name "Name" git config --global user.email "email" git config --global init.defaultBranch main git config --global core.editor "code --wait" git config --list

Starting a repository

git init Create a new repository git clone URL Clone a remote repository

Daily workflow

git status Check current state git diff Show unstaged changes git diff --staged Show staged changes git add filename Stage a file git add . Stage all changes git commit -m "message" Commit staged changes git log --oneline View commit history git log --oneline --graph --all Visual branch history

Undoing things

git restore filename Discard working tree changes git restore --staged filename Unstage a file git commit --amend Fix last commit git revert SHA Safe undo — adds new commit git reset --soft SHA Erase commits, keep staged git reset --mixed SHA Erase commits, keep unstaged git reset --hard SHA Erase commits and changes git reflog View full HEAD history

Branching

git branch List branches git branch name Create a branch git switch name Switch to a branch git switch -c name Create and switch git merge name Merge branch into current git branch -d name Delete merged branch git branch -D name Force delete branch git merge --abort Cancel a merge

Remotes

git remote add origin URL Connect to remote git remote -v Show remotes git push -u origin main First push with tracking git push Push commits git pull Fetch and merge git fetch Fetch without merging

Advanced

git stash Shelve changes git stash pop Restore shelved changes git cherry-pick SHA Copy a commit git bisect start Start binary search git blame filename Who changed each line git rebase -i HEAD~N Interactive rebase

---

Chapter Summary

Practice	Why it matters

Great commit messages	Future you and your team can understand the history
Small focused commits	Easy to review, revert and understand
Branch for everything	Main stays clean and deployable
Pull before pushing	Avoid rejected pushes and conflicts
Never commit secrets	Secrets in repos cannot be truly erased
Review code carefully	Catch bugs before they reach production
Build your profile	GitHub is your portfolio — keep it active

Final Assessment — The Complete Book Review

Question 1 Describe the complete GitHub Flow from creating a branch to the code being live in production.

Question 2 You join a new team and notice their commit messages look like: `feat(auth): add JWT token refresh endpoint` What convention are they using and what are its benefits?

Question 3 A junior developer on your team accidentally committed their AWS credentials to a public GitHub repository 10 minutes ago. What is the correct sequence of actions?

Question 4 Your team has been working on a feature for three weeks. The feature branch has 47 commits — most of them are "wip", "fix", "more fixes". Before merging the PR, how do you clean this up?

Question 5 You are a solo developer building a personal project. Make the case for STILL using branches and pull requests even though no one else will review your code.

Final Answers

Answer 1 Complete GitHub Flow:

1. Create a branch from main: `git switch -c feature/name`
2. Make commits on the branch: `git add . && git commit -m "..."`
3. Push the branch: `git push origin feature/name`
4. Open a pull request on GitHub with title and description
5. CI runs automatically — tests and checks must pass
6. Teammates review the code — leave comments, request changes
7. Author addresses feedback — pushes new commits to the branch
8. Reviewers approve the PR
9. PR is merged into main (squash, merge commit or rebase)
10. Branch is deleted on GitHub and locally
11. GitHub Actions detects the push to main
12. Deployment workflow runs automatically
13. Code is live in production within minutes

Answer 2 They are using the Conventional Commits specification.

Benefits:

- Structured, machine-readable commit history
- Automated changelog generation from commit types

- Clear communication of the nature of each change
- Semantic versioning can be automated from commit types
- `feat` = minor version bump, `fix` = patch, breaking change = major
- Easier to filter and search commit history by type

Answer 3 Immediate actions in order:

1. Rotate the credentials immediately in AWS — generate new keys and deactivate the exposed ones. Do this FIRST — every second counts.
2. Delete the file from the working tree and commit the removal
3. Add the file to `.gitignore`
4. Rewrite Git history to remove the file from all commits (use BFG Repo Cleaner — faster and safer than filter-branch)
5. Force push the rewritten history to GitHub
6. Contact GitHub support to request a cache clear
7. Audit AWS logs for any unauthorized access during the exposure window
8. Notify the team and document what happened and how it was resolved

Answer 4 Use interactive rebase to squash the 47 commits:

git rebase -i HEAD~47

In the editor:

- Keep the first commit as `pick` with a proper message
- Change all remaining 46 commits to `fixup`

This combines all 47 commits into one clean commit with a professional message that describes the complete feature.

If 47 is too many for one rebase, do it in stages: rebase 20 at a time, squashing as you go.

Answer 5 The case for branches and PRs as a solo developer:

Safety — broken code on a branch cannot break your live site. You can abandon a branch without any consequences to main.

Review opportunity — switching from a branch to the PR view on GitHub forces you to look at your changes with fresh eyes. You will catch mistakes you missed while writing the code.

History — the commit log becomes a journal of decisions. Six months later you will understand why you made each change.

Habit building — the developers who thrive in teams are those who practiced good habits before joining one. Solo projects are the training ground.

Rollback — if a feature causes problems, reverting a single merged PR is clean and simple. Reverting direct commits to main is messier.

Portfolio — recruiters and collaborators look at how you work, not just what you built. A clean branching history with descriptive PR descriptions signals professional standards.

Congratulations

You have completed Git and GitHub — From Zero to Professional.

You started with nothing and now you have:

- A complete understanding of how Git works internally
- Hands-on experience with every important Git command
- The ability to collaborate professionally using pull requests
- A working CI/CD pipeline using GitHub Actions
- A live website deployed on GitHub Pages
- A professional GitHub profile showcasing your work

Git is a tool. Like all tools, it gets easier with use. The concepts in this book will become second nature as you apply them every day.

Keep committing. Keep pushing. Keep learning.

— End of Book —

markdown

Annex A — Terminal and Markdown Quick Reference

About This Annex

This annex is a practical reference card for two essential tools used throughout this book — the terminal and Markdown.

Bookmark this page. Return to it whenever you need a quick reminder of a command or syntax.

Part 1 — Terminal Commands

What is the Terminal?

The terminal (also called Command Prompt on Windows, or Terminal on Mac and Linux) is a text-based interface for communicating with your computer.

Instead of clicking with a mouse, you type commands. Git is used entirely through the terminal.

1.1 Windows Command Prompt

Navigation

Command	What it does	Example
`cd foldername`	Move into a folder	`cd Desktop`
`cd ..`	Go up one folder level	`cd ..`
`cd C:\full\path`	Go to a specific path	`cd C:\Users\Hassan\Desktop`
`cd ~`	Go to home directory	`cd ~`
`dir`	List files and folders	`dir`
`dir /a`	List including hidden files	`dir /a`

Creating and deleting

Command	What it does	Example
`mkdir foldername`	Create a new folder	`mkdir my-project`
`echo text > file`	Create a file with content	`echo Hello > readme.txt`
`echo text >> file`	Append text to a file	`echo World >> readme.txt`
`del filename`	Delete a file	`del readme.txt`
`rmdir foldername`	Delete an empty folder	`rmdir old-folder`

`rmdir /s foldername`	Delete folder and contents	`rmdir /s old-folder`

Reading files

Command	What it does	Example
`type filename`	Print file contents	`type readme.txt`
`more filename`	Print file page by page	`more longfile.txt`

Useful shortcuts

Shortcut	What it does
`Tab`	Auto-complete folder and file names
`Up arrow`	Repeat previous command
`Ctrl+C`	Stop a running command
`cls`	Clear the terminal screen
`Esc`	Clear the current line
`F7`	Show command history

1.2 Mac / Linux Terminal

Navigation

Command	What it does	Example
`cd foldername`	Move into a folder	`cd Desktop`
`cd ..`	Go up one folder level	`cd ..`
`cd ~/path`	Go to path from home	`cd ~/Desktop/my-project`
`cd ~`	Go to home directory	`cd ~`
`ls`	List files and folders	`ls`
`ls -la`	List including hidden files	`ls -la`
`pwd`	Print current directory path	`pwd`

Creating and deleting

Command	What it does	Example
`mkdir foldername`	Create a new folder	`mkdir my-project`
`mkdir -p path`	Create nested folders	`mkdir -p .github/workflows`
`echo "text" > file`	Create file with content	`echo "Hello" > readme.txt`

`echo "text" >> file`	Append text to file	`echo "World" >> readme.txt`
`touch filename`	Create an empty file	`touch index.html`
`rm filename`	Delete a file	`rm readme.txt`
`rm -rf foldername`	Delete folder and all contents	`rm -rf old-folder`

Reading files

Command	What it does	Example
`cat filename`	Print file contents	`cat readme.txt`
`less filename`	Print file page by page	`less longfile.txt`
`head filename`	Print first 10 lines	`head readme.txt`
`tail filename`	Print last 10 lines	`tail readme.txt`

Useful shortcuts

Shortcut	What it does
`Tab`	Auto-complete folder and file names
`Up arrow`	Repeat previous command
`Ctrl+C`	Stop a running command
`clear`	Clear the terminal screen
`Ctrl+L`	Clear the terminal screen
`Ctrl+A`	Jump to start of line
`Ctrl+E`	Jump to end of line

1.3 Understanding File Paths

Windows paths

C:\Users\Hassan\Desktop\my-project\index.html | | | | | | | | | | | file | | | | folder | | | folder | | username | separator drive letter

Mac / Linux paths

/home/hassan/Desktop/my-project/index.html | | | | | | | | | file | | | folder | | folder | username root

Relative vs absolute paths

Absolute — full path from root: C:\Users\Hassan\Desktop\my-project (Windows) /home/hassan/Desktop/my-project (Mac/Linux)

Relative — path from current location: my-project (if you are already in Desktop) ./my-project (same thing, explicit) ../my-project (go up one level then into my-project)

1.4 The > and >> Operators

These operators redirect output to a file.

Operator	Behaviour	Use case
>	Creates file or **overwrites** existing	Starting fresh
>>	Creates file or **appends** to existing	Adding content

echo Line 1 > file.txt ← creates file with "Line 1" echo Line 2 > file.txt ← OVERWRITES — now only "Line 2" echo Line 3 >> file.txt ← APPENDS — now has "Line 2" and "Line 3"

1.5 PowerShell Differences (Windows VS Code Terminal)

VS Code on Windows opens PowerShell by default, not Command Prompt. Most commands are the same but a few differ:

Action	Command Prompt	PowerShell
Create file	`echo text > file`	`New-Item filename`
List files	`dir`	`ls` or `dir`
Clear screen	`cls`	`clear` or `cls`
Print file	`type filename`	`cat filename`

Part 2 — Markdown Reference

What is Markdown?

Markdown is a simple formatting language that converts plain text to formatted output. GitHub renders Markdown files (`.md`) as beautifully formatted pages automatically.

You write plain text with simple symbols. GitHub does the rest.

2.1 Headings

```
# Heading 1 — largest
## Heading 2
### Heading 3
#### Heading 4
##### Heading 5
###### Heading 6 — smallest
```

Rendered as:

- `#` = large bold heading
- `##` = medium heading
- `###` = smaller heading

2.2 Text Formatting

Markdown	Result
`**bold text**`	**bold text**
`*italic text*`	*italic text*
`***bold and italic***`	***bold and italic***
`~~strikethrough~~`	~~strikethrough~~
`` `inline code` ``	`inline code`

2.3 Lists

Unordered list

```
- Item one
- Item two
- Item three
  - Nested item
  - Another nested item
```

Also works with `*` or `+` instead of `-`.

Ordered list

```
1. First item
2. Second item
3. Third item
```

Task list (checkboxes)

```
- [x] Completed task
- [ ] Incomplete task
- [ ] Another task
```

Renders as interactive checkboxes on GitHub.

2.4 Links

Basic link

```
[Link text](https://example.com)
```

Link with title (tooltip on hover)

```
[Link text](https://example.com "Tooltip text")
```

Link to a section in the same document

```
[Go to Chapter 2](#chapter-2-the-language-of-git)
```

Bare URL (auto-linked)

```
https://github.com
```

2.5 Images

```
![Alt text](path/to/image.jpg)
![Alt text](https://example.com/image.jpg)
```

Image with link

```
[![Alt text](image.jpg)](https://example.com)
```

Controlling image size (HTML in Markdown)

```
<img src="image.jpg" alt="Description" width="300">
```

2.6 Code

Inline code

```
Use `git status` to check the state.
```

Code block (fenced)

````
```
plain code block
````
````

````
no syntax highlighting
```
````

Code block with syntax highlighting

````
```python
def greet(name):
    return f"Hello, {name}!"
```
````

````
```javascript
function greet(name) {
  return `Hello, ${name}!`;
}
```
````

````
```bash
git add .
git commit -m "Add feature"
git push
```
````

````
```yaml
name: My Workflow
on: [push]
```
````

Common language identifiers: `python`, `javascript`, `html`, `css`, `bash`, `yaml`, `json`, `sql`, `markdown`, `java`, `c`, `cpp`

2.7 Tables

```
| Column 1 | Column 2 | Column 3 |
|----------|----------|----------|
| Row 1    | Data     | Data     |
| Row 2    | Data     | Data     |
| Row 3    | Data     | Data     |
```

Column alignment

```
| Left | Centre | Right |
|:-----|:------:|------:|
| text | text   | text  |
```

- `:---` = left aligned (default)
```

- `:---:` = centre aligned
- `---:` = right aligned

2.8 Blockquotes

```
> This is a blockquote.
> It can span multiple lines.

> Nested blockquote:
>> This is nested inside the first quote.
```

2.9 Horizontal Rule

Three or more hyphens, asterisks or underscores:

```
---
***
___
```

All three render as a horizontal dividing line.

2.10 Escaping Special Characters

If you want to display a character that Markdown uses for formatting, escape it with a backslash:

```
\*not italic\*
\# not a heading
\` not code \`
\[not a link\]
```

2.11 HTML in Markdown

GitHub Markdown supports basic HTML tags when you need more control:

```
<br>                    Line break
<strong>bold</strong>   Bold text
<em>italic</em>         Italic text
<details>               Collapsible section
<summary>Title</summary>
Content here
</details>
```

Collapsible section example

```
<details>
<summary>Click to expand</summary>

Content that is hidden until clicked.
You can put any Markdown here.

</details>
```

2.12 GitHub-Specific Markdown

GitHub adds extra features beyond standard Markdown.

Mentioning people

```
@username
```

Tags a GitHub user — they receive a notification.

Referencing issues and PRs

```
#42           Reference issue or PR number 42
Closes #42    Close issue 42 when this PR merges
Fixes #42     Same as Closes
```

Status badges

```
![Build Status](https://github.com/USER/REPO/actions/workflows/main.yml/badge.svg)
```

Emoji

```
:rocket: :tada: :white_check_mark: :x: :warning:
```

Renders as: 🚀 🎉 ✅ ❌ ⚠️

2.13 Complete README Template

A README that follows best practices:

```
# Project Name

Brief one-sentence description of what this project does.

![Build Status](badge-url-here)

## Overview
```

```
2-3 sentences explaining the problem this solves and
who it is for.

## Features

- Feature one
- Feature two
- Feature three

## Installation

\```bash
git clone https://github.com/username/project.git
cd project
npm install
\```

## Usage

\```bash
npm start
\```

Example:
\```javascript
const result = doSomething('input');
console.log(result);
\```

## Configuration

| Variable | Default | Description |
|----------|---------|-------------|
| `PORT` | `3000` | Server port |
| `DEBUG` | `false` | Enable debug mode |

## Contributing

1. Fork the repository
2. Create a feature branch: `git switch -c feature/name`
3. Commit your changes: `git commit -m "Add feature"`
4. Push to your fork: `git push origin feature/name`
5. Open a pull request

## Licence

MIT Licence — see [LICENSE](LICENSE) for details.

## Author
```

```
Your Name — [your-website.com](https://your-website.com)
GitHub: [@yourusername](https://github.com/yourusername)
```

2.14 Markdown Editors and Previews

VS Code

Press `Ctrl+Shift+V` to open a Markdown preview panel. Or click the preview icon in the top right of the editor.

Online editors

Tool	URL
StackEdit	stackedit.io
Dillinger	dillinger.io
HackMD	hackmd.io

GitHub preview

Every `.md` file on GitHub has a **Preview** button at the top of the file view. Click it to see the rendered output.

Quick Reference Card

Terminal — most used commands

cd foldername move into folder cd .. go up one level ls / dir list files mkdir foldername create folder echo text > file create file echo text >> file append to file cat / type filename read file clear / cls clear screen Tab autocomplete Up arrow repeat last command Ctrl+C stop command

Markdown — most used syntax

Heading 1 Heading 2

bold italic inline code link text Show Image

bullet point numbered list checkbox

blockquote --- horizontal rule

Code block languages

python javascript html css bash yaml json sql

End of Annex A

Annex B — Git Cheat Sheet

Your One-Page Git Reference

Print this page and keep it at your desk. Every command you will ever need — organised by what you are trying to do.

SETUP — Do this once per machine

git config --global user.name "Your Name" git config --global user.email "you@example.com" git config --global init.defaultBranch main git config --global core.editor "code --wait" git config --list

---

START — Beginning a project

git init Start tracking a folder with Git git clone URL Download a repo from GitHub git remote add origin URL Connect local repo to GitHub git remote -v Verify remote connection

---

DAILY WORKFLOW — What you do every day

git status What has changed? git diff Show exact line changes (unstaged) git diff --staged Show staged changes git add filename Stage one file git add . Stage everything git commit -m "message" Save a snapshot git log --oneline View history (compact) git log --oneline --graph --all Visual branch history git push Upload commits to GitHub git pull Download and apply changes

---

BRANCHES — Parallel lines of work

git branch List all branches git branch name Create a branch git switch name Move to a branch git switch -c name Create AND move in one step git merge name Merge branch into current git branch -d name Delete a merged branch git branch -D name Force delete any branch git push origin name Push a branch to GitHub git merge --abort Cancel a merge in progress

---

UNDOING — Fixing mistakes

git restore filename Discard working tree changes git restore --staged filename Unstage a file git commit --amend -m "msg" Fix last commit message git revert SHA Safe undo — adds new commit git reset --soft SHA Erase commits, keep staged git reset --mixed SHA Erase commits, keep unstaged git reset --hard SHA Erase commits AND changes

---

RESCUE — When things go wrong

git reflog Full history of HEAD movements git stash Shelve uncommitted work git stash pop Restore shelved work git stash list Show all stashes git cherry-pick SHA Copy one commit to current branch git bisect start Start binary search for a bug git bisect good SHA Mark a known good commit git bisect bad Mark current commit as bad git bisect reset End bisect session git blame filename Who changed each line

---

REMOTE — Working with GitHub

git remote add origin URL Connect to GitHub git remote -v Show remotes git remote remove name Remove a remote git push -u origin main First push (sets tracking) git push Push to tracked remote git pull Fetch and merge git fetch Fetch without merging git clone URL Download a complete repo git branch -vv Show tracking relationships

---

HISTORY — Reading the past

git log Full commit history git log --oneline Compact history git log --oneline --graph --all Visual branch graph git log --author="Name" Commits by one author git log --since="2 weeks ago" Recent commits git log -- filename History of one file git diff SHA1 SHA2 Compare two commits git show SHA Show one commit in full

---

ADVANCED — Power user commands

git rebase -i HEAD~N Interactive rebase — clean history git rebase branch Rebase onto another branch git tag v1.0.0 Create a tag git tag -a v1.0.0 -m "Release" Annotated tag git push origin --tags Push all tags git rm --cached filename Stop tracking a file git clean -fd Remove untracked files git submodule add URL Add a submodule

---

SHORTCUTS — Save time every day

Set these aliases once and use them forever:

git config --global alias.st status git config --global alias.co checkout git config --global alias.br branch git config --global alias.ci commit git config --global alias.lg "log --oneline --graph --all" git config --global alias.last "log -1 HEAD" git config --global alias.unstage "restore --staged" git config --global alias.undo "reset HEAD~1 --mixed"

After setting:

git st instead of git status git lg instead of git log --oneline --graph --all git last instead of git log -1 HEAD git unstage instead of git restore --staged git undo instead of git reset HEAD~1 --mixed

---

THE THREE ZONES — Never forget this

Zone 1 Zone 2 Zone 3 Working Tree --> Staging Area --> Repository (your files) (the basket) (permanent history)

git add git commit

git restore git restore --staged (discard) (unstage)

COMMIT MESSAGE FORMAT

Good: Add user login page Good: Fix null pointer in payment processing Good: Update README with installation steps Good: Remove deprecated getUserData function

Bad: fix Bad: update Bad: wip Bad: changes Bad: asdfgh

The rule: Complete this sentence — "If applied, this commit will..."

WHEN THINGS GO WRONG — Quick fixes

Problem	Fix

Wrong commit message	`git commit --amend -m "correct message"`
Committed to wrong branch	`git reset HEAD~1` then switch branch
Need to undo last commit	`git reset --soft HEAD~1`
Accidentally deleted work	`git reflog` then `git reset --hard SHA`
Push rejected	`git pull` then `git push`
Merge conflict	Edit files, remove markers, `git add`, `git commit`
Forgot to stage a file	`git add file` then `git commit --amend --no-edit`
Stashed work disappeared	`git stash list` then `git stash apply stash@{N}`

THE EVERYDAY WORKFLOW IN 6 STEPS

git status Check where you are git diff See what changed git add . Stage your work git diff --staged Verify what will be committed git commit -m "msg" Save the snapshot git push Send to GitHub

Repeat. Every day. It becomes muscle memory.

GITHUB FLOW IN 7 STEPS

git switch -c feature/name Create a branch [make your changes] Do the work git add . && git commit Commit regularly git push origin feature/name Push the branch [open pull request on GitHub] Request review [review and approve] Team checks the work [merge and delete branch] Ship it

---

Keep committing. Keep pushing. Keep learning.

— Git and GitHub: From Zero to Professional — Dr M Quamrul Hassan

Annex C — Troubleshooting Guide

About This Guide

This guide covers the 25 most common Git problems — the exact error messages you will see and the exact steps to fix them.

When something goes wrong, find your error message here and follow the steps. Every problem has a solution.

How to Use This Guide

1. Find your error message in the section headings
2. Read the cause — understand why it happened
3. Follow the fix — step by step
4. Read the prevention — avoid it next time

Problem 1 — git is not recognised

Error message:

'git' is not recognized as an internal or external command, operable program or batch file.

Cause: Git is not installed, or the terminal window was open before Git was installed.

Fix:

Step 1: Close ALL terminal windows completely Step 2: Open a brand new terminal Step 3: Type: git --version

If still not recognised:

Step 4: Restart your computer Step 5: Type: git --version again

If still failing — Git was not installed correctly. Reinstall from https://git-scm.com

Prevention: Always open a fresh terminal after installing new software.

Problem 2 — Push rejected (fetch first)

Error message:

! [rejected] main -> main (fetch first) error: failed to push some refs to 'https://github.com/...' hint: Updates were rejected because the remote contains hint: work that you do not have locally.

Cause: Someone else (or you on another machine) pushed commits to GitHub since your last pull. Your local history is behind the remote.

Fix:

git pull git push

If pull creates a merge conflict — resolve it, then push.

Prevention: Always `git pull` before starting work each session.

Problem 3 — Push rejected (non-fast-forward)

Error message:

! [rejected] main -> main (non-fast-forward) error: failed to push some refs hint: Updates were rejected because the tip of your hint: current branch is behind its remote counterpart.

Cause: Same as Problem 2 — remote has commits you do not have.

Fix:

git pull --rebase git push

`--rebase` replays your commits on top of the remote commits — keeps history cleaner than a merge.

Problem 4 — Remote origin already exists

Error message:

error: remote origin already exists.

Cause: You ran `git remote add origin` but a remote called `origin` already exists for this repo.

Fix:

git remote remove origin git remote add origin YOUR-CORRECT-URL git remote -v

Verify the new remote is correct before pushing.

Problem 5 — Authentication failed

Error message:

remote: Invalid username or password. fatal: Authentication failed for 'https://github.com/...'

Cause: GitHub no longer accepts your account password for terminal operations. You need a Personal Access Token.

Fix:

Step 1: Go to GitHub → Profile → Settings → Developer settings Step 2: Personal access tokens → Tokens (classic) Step 3: Generate new token (classic) Step 4: Select 'repo' scope → Generate Step 5: Copy the token (starts with ghp_) Step 6: When Git asks for password — paste the token

Prevention: Set up Git Credential Manager to store your token:

git config --global credential.helper manager

Problem 6 — Repository not found

Error message:

ERROR: Repository not found. fatal: Could not read from remote repository.

Cause: Wrong URL, the repository is private and you are not authenticated, or the repository was deleted.

Fix:

Step 1: Check the URL is correct git remote -v Step 2: If wrong URL: git remote remove origin git remote add origin CORRECT-URL Step 3: If private repo — authenticate properly (see Problem 5 for token setup)

Problem 7 — Merge conflict

Error message:

CONFLICT (content): Merge conflict in filename.txt Automatic merge failed; fix conflicts and then commit the result.

Cause: Two branches made different changes to the same lines of the same file.

Fix:

Step 1: Open the conflicted file Step 2: Find the conflict markers: <<<<<<< HEAD your version

their version

branch-name Step 3: Edit the file — keep what you want, remove markers Step 4: git add filename.txt Step 5: git commit

To cancel and go back to before the merge:

git merge --abort

Prevention: Pull frequently. Small focused branches reduce conflicts.

Problem 8 — Detached HEAD

Error message:

HEAD detached at a3f9c2b

Cause: You checked out a specific commit SHA instead of a branch. You are not on any branch — commits made here may be lost.

Fix: If you want to keep work done in detached HEAD state:

git switch -c new-branch-name

If you just want to go back to main:

git switch main

Prevention: Always check out branch names, not commit SHAs, unless you are specifically browsing history.

Problem 9 — Nothing to commit

Message:

nothing to commit, working tree clean

Cause: This is not an error — it means everything is committed. No uncommitted changes exist.

If you expected changes:

Step 1: git status Check what Git sees Step 2: git diff Check for unstaged changes Step 3: Check you saved the file in your editor Step 4: Check you are in the right folder

---

Problem 10 — Untracked files after adding to .gitignore

Problem: You added a file to `.gitignore` but `git status` still shows it as modified or tracked.

Cause: `.gitignore` only ignores files that have never been tracked. Once a file is committed, adding it to `.gitignore` does not un-track it.

Fix:

git rm --cached filename git commit -m "Stop tracking filename"

The file stays on your disk but Git stops watching it.

Prevention: Add entries to `.gitignore` before creating the files.

Problem 11 — Accidentally committed to main

Problem: You made commits directly to main that should have been on a feature branch.

Fix (commits not yet pushed):

Step 1: Create the correct branch pointing here git switch -c feature/correct-branch Step 2: Go back to main git switch main Step 3: Remove the commits from main git reset --hard origin/main Step 4: Your commits are now safely on feature/correct-branch

Fix (commits already pushed):

Step 1: Revert each bad commit git revert SHA Step 2: Push the reversions git push

---

Problem 12 — Wrong commit message

Problem: You typed the wrong commit message.

Fix (last commit, not yet pushed):

git commit --amend -m "Correct message here"

Fix (last commit, already pushed):

git revert SHA git commit -m "Correct action with right message"

Never amend pushed commits — it causes problems for others.

Problem 13 — Accidentally deleted a branch

Problem: You ran `git branch -d` or `git branch -D` and deleted a branch you still needed.

Fix:

Step 1: Find the last commit on the deleted branch git reflog Step 2: Look for the commit SHA before the deletion Step 3: Recreate the branch pointing to that commit git switch -c recovered-branch SHA

Prevention: Always verify a branch is merged before deleting. Use `-d` (safe delete) not `-D` (force delete) by default.

Problem 14 — Lost work after git reset --hard

Problem: You ran `git reset --hard` and lost commits you needed.

Fix:

Step 1: git reflog Step 2: Find the SHA of the commit before the reset (look for HEAD@{1} or similar) Step 3: git reset --hard THAT-SHA

Your commits are restored.

Prevention: Before any destructive operation, note the current SHA:

git log --oneline -1

Problem 15 — Cannot switch branches (unsaved changes)

Error message:

error: Your local changes to the following files would be overwritten by checkout: filename.txt Please commit your changes or stash them before you switch branches.

Cause: You have uncommitted changes that would be overwritten by switching branches.

Fix: Option 1 — Stash the changes:

git stash git switch other-branch [do your work] git switch back git stash pop

Option 2 — Commit the changes:

git add . git commit -m "WIP - switching branches" git switch other-branch

Option 3 — Discard the changes:

git restore . git switch other-branch

Problem 16 — Diverged branches

Error message:

Your branch and 'origin/main' have diverged, and have 1 and 1 different commits each, respectively.

Cause: Your local branch and the remote branch both have commits the other does not have. They have diverged.

Fix:

git pull --rebase git push

Or if you prefer a merge commit:

git pull git push

Problem 17 — File too large for GitHub

Error message:

remote: error: File filename is 150.00 MB; this exceeds GitHub's file size limit of 100 MB.

Cause: GitHub rejects files larger than 100MB.

Fix:

Step 1: Remove the file from the last commit git rm --cached large-file Step 2: Add it to .gitignore echo large-file >> .gitignore Step 3: Commit the removal git commit --amend --no-edit Step 4: Push git push

For very large files use Git LFS (Large File Storage):

git lfs track "*.psd" git add .gitattributes

Problem 18 — SSL certificate error

Error message:

SSL certificate problem: unable to get local issuer certificate

Cause: Your network or VPN is intercepting HTTPS connections. Common in corporate environments.

Fix (temporary):

git config --global http.sslVerify false

Better fix: Contact your IT department for the correct SSL certificate to add to Git's trust store.

Prevention: Only disable SSL verification as a last resort and re-enable it when done.

Problem 19 — Line ending warnings

Warning message:

warning: LF will be replaced by CRLF in filename. The file will have its original line endings in your working directory.

Cause: Windows uses CRLF line endings. Mac/Linux uses LF. Git is warning you about the conversion.

Fix (Windows — recommended):

git config --global core.autocrlf true

Fix (Mac/Linux — recommended):

git config --global core.autocrlf input

This is a warning not an error — your files will work correctly.

Problem 20 — Accidentally staged everything

Problem: You ran `git add .` and staged files you did not want to include in this commit.

Fix — unstage everything:

git restore --staged .

Fix — unstage one file:

git restore --staged filename

Your changes are preserved in the working tree — only the staging is undone.

Problem 21 — Cannot push to protected branch

Error message:

remote: error: GH006: Protected branch update failed remote: error: Required status check "ci" is failing.

Cause: The branch has protection rules — direct pushes are blocked, or required checks have not passed.

Fix:

Step 1: Create a feature branch git switch -c feature/my-change Step 2: Push the feature branch git push origin feature/my-change Step 3: Open a pull request on GitHub Step 4: Wait for required checks to pass Step 5: Get required approvals Step 6: Merge via GitHub

---

Problem 22 — Submodule errors

Error message:

fatal: No url found for submodule path 'subfolder'

Cause: The repository contains submodules that have not been initialised.

Fix:

git submodule update --init --recursive

When cloning a repo with submodules:

git clone --recurse-submodules URL

Problem 23 — git log shows nothing

Problem: `git log` returns immediately with no output.

Cause: No commits have been made yet in this repository.

Fix: Make your first commit:

git add . git commit -m "Initial commit"

Problem 24 — Wrong email in commits

Problem: Your commits show the wrong email address — they do not link to your GitHub profile.

Fix for future commits:

git config --global user.email "correct@email.com"

Fix for the last commit:

git commit --amend --reset-author --no-edit

Fix for multiple past commits: Use `git rebase -i` to rewrite history. Warning: Only do this for commits not yet pushed.

Problem 25 — Workflow file not running

Problem: You pushed a `.github/workflows/main.yml` file but the workflow is not appearing in the Actions tab.

Cause: YAML syntax error, wrong file location, or wrong trigger.

Fix:

Step 1: Check the file is in exactly: .github/workflows/main.yml (not .github/workflow/ or github/workflows/)

Step 2: Validate YAML syntax at: https://yamlint.com

Step 3: Check the trigger matches your branch name: on: push: branches: [main] (not master if your branch is main)

Step 4: Check the Actions tab for any error messages from GitHub about the workflow file

---

Quick Diagnosis Checklist

When anything goes wrong run these in order:

git status What is the current state? git log --oneline Where is HEAD? git remote -v Is the remote correct? git branch Which branch am I on? git diff What has changed? git reflog What has HEAD done recently?

These six commands will reveal the cause of almost every Git problem.

Getting More Help

GitHub documentation

https://docs.github.com/en/get-started/using-git

Oh Shit Git — plain English fixes

https://ohshitgit.com

Stack Overflow

Search for your exact error message — someone has almost certainly solved it before.

Git official documentation

https://git-scm.com/doc

End of Annex C

Remember: In Git, almost nothing is truly lost. If you committed it, reflog can find it. Stay calm, diagnose carefully, fix methodically.

Annex D — Glossary

Complete Git and GitHub Terminology

All terms used in this book listed alphabetically. Use this as a quick reference when you encounter an unfamiliar term.

A

Alias

A custom shortcut for a longer Git command. Set with `git config --global alias.name "command"`. For example `git config --global alias.st status` lets you type `git st` instead of `git status`.

Annotated tag

A tag that stores extra metadata — tagger name, email, date and a message. Created with `git tag -a v1.0 -m "msg"`. Recommended for release versions. Compare with lightweight tag.

Artifact

In GitHub Actions, a file or collection of files produced during a workflow run and saved for later use. Created with the `actions/upload-artifact` action.

B

Base branch

In a pull request, the branch that changes will be merged INTO. Usually `main`. Compare with compare branch.

Binary search

The algorithm used by `git bisect` to find a bad commit. Halves the search space at each step — 200 commits takes maximum 8 steps to find the culprit.

Bisect

A Git command (`git bisect`) that uses binary search to find exactly which commit introduced a bug. You mark commits as good or bad and Git narrows it down.

Blame

A Git command (`git blame`) that shows who last changed each line of a file, when, and in which commit.

Branch

A lightweight pointer to a specific commit — an independent line of development. Creating a branch takes milliseconds because Git only creates a pointer, not a copy of files.

Branch protection

GitHub rules preventing direct pushes to important branches. Configured in Repository → Settings → Branches. Can require pull requests, approvals and passing checks before merging.

C

Cherry-pick

A Git command (`git cherry-pick SHA`) that copies the changes from one specific commit and applies them as a new commit on the current branch.

CI/CD

Continuous Integration and Continuous Deployment. Automated processes that run tests on every push (CI) and deploy code automatically when tests pass (CD).

Clone

Downloading a complete copy of a remote repository — including all files, all commits and all branches — to your local machine. Done with `git clone URL` .

Commit

A permanent snapshot of your project at a specific moment. Contains a unique SHA hash, author, timestamp, message and a record of all changes.

Commit message

A description of what a commit does and why. Written with `git commit -m "message"` . Good messages complete the sentence "If applied, this commit will..."

Compare branch

In a pull request, the branch containing the new changes being proposed. Compare with base branch.

Conflict markers

The symbols Git inserts into a file during a merge conflict: `<<<<<<<` , `=======` and `>>>>>>>` . You must remove these markers after resolving the conflict.

Conventional Commits

A commit message specification using a structured format: `type(scope): description` . Types include `feat` , `fix` , `docs` , `refactor` , `test` and `chore` .

D

Default branch

The primary branch of a repository. Modern standard is `main` . Older repositories may use `master` . Configured with `git config --global init.defaultBranch main` .

Detached HEAD

A state where HEAD points directly to a commit rather than to a branch. Commits made in detached HEAD state may be lost unless you create a branch. Fix with `git switch -c name` .

Diff

The difference between two versions of a file or two commits. Lines starting with `+` were added (green), lines starting with `-` were removed (red). Shown with `git diff`.

Draft pull request

A pull request marked as work in progress — not ready for review yet. Converted to a regular PR when ready by clicking "Ready for review".

E

Environment variable

A configuration value passed to a program. In GitHub Actions, used to pass configuration to workflow steps. Never hardcode secrets — use GitHub Secrets instead.

F

Fast-forward merge

A merge that occurs when the base branch has no new commits since the feature branch was created. Git simply moves the branch pointer forward — no new commit is created.

Fetch

Downloading changes from a remote repository without merging them. Your working files remain unchanged. Use `git fetch` to review incoming changes before integrating.

Fork

A personal copy of someone else's GitHub repository, created on GitHub's servers. You have full write access to your fork. Used for contributing to projects you do not own.

Force push

Pushing to a remote even when the histories have diverged. Done with `git push --force`. Dangerous on shared branches — rewrites remote history and causes problems for teammates.

G

Git

A free, open-source distributed version control system created by Linus Torvalds in 2005. Runs on your local machine. Tracks changes to files over time.

GitHub

A web platform that hosts Git repositories online. Owned by Microsoft. Adds collaboration features like pull requests, issues and GitHub Actions on top of Git.

GitHub Actions

GitHub's built-in CI/CD platform. Runs automated workflows defined in YAML files when Git events occur — such as pushing or opening a pull request.

GitHub Flow

The most widely used team workflow. Branch from main → commit → push → pull request → review → merge → delete branch. Main is always deployable.

GitHub Pages

A free static website hosting service built into GitHub. Serves HTML, CSS and JavaScript files directly from a repository at `username.github.io`.

.gitignore

A file in the root of a repository listing files and patterns that Git should never track. Commonly used to exclude secrets, dependencies and build output.

GitFlow

A branching strategy for versioned software with scheduled releases. Uses separate `main`, `develop`, `feature`, `release` and `hotfix` branches.

H

HEAD

A special pointer showing your current position in the repository. Usually points to the tip of your current branch. Moves forward automatically when you commit.

History

The complete record of all commits in a repository — who made them, when, and what changed. Viewed with `git log`.

Hook

A script that runs automatically at specific points in the Git workflow — before committing, after pushing etc. Stored in `.git/hooks/`. Used to enforce standards.

Hotfix

An emergency fix applied directly to a production branch. In GitFlow, created from `main` and merged back to both `main` and `develop`. Should be small and targeted.

I

Index

Another name for the staging area. The intermediate zone between your working tree and the repository where changes are prepared before committing.

Interactive rebase

A rebase mode (`git rebase -i HEAD~N`) that lets you rewrite, squash, reorder or delete commits before pushing. Used to clean up messy history before opening a pull request.

J

Job

In GitHub Actions, a set of steps that run on one virtual machine. Multiple jobs in a workflow run in parallel by default. Use `needs` to create dependencies between jobs.

L

Lightweight tag

A simple named pointer to a commit with no extra metadata. Created with `git tag v1.0` . Compare with annotated tag.

Local repository

The copy of a repository on your own machine. Contains the full history. You commit here before pushing to remote.

M

main

The default primary branch in modern Git repositories. Replaced `master` as the standard default. Always kept in a deployable state in professional workflows.

Markdown

A lightweight formatting language that converts plain text to formatted output. GitHub renders `.md` files automatically. Used for README files and documentation.

Merge

Combining the changes from one branch into another. Creates a merge commit in a 3-way merge, or simply moves a pointer in a fast-forward merge.

Merge commit

A special commit created during a 3-way merge that has two parent commits — one from each branch being merged. Preserves the branching history.

Merge conflict

When two branches have made different changes to the same lines of the same file and Git cannot automatically merge them. Must be resolved manually.

master

The old default branch name, replaced by `main` as the modern standard. Still used in older repositories. Functionally identical to `main` .

O

Origin

The conventional name for the primary remote — the one you cloned from or connected to. Just a shortcut name for the remote URL. Set with `git remote add origin URL`.

P

Patch mode

An interactive staging mode (`git add -p`) that lets you stage individual chunks of a file rather than the whole file. Useful when a file contains multiple unrelated changes.

Personal Access Token (PAT)

A token used instead of a password for GitHub authentication in the terminal. Created at GitHub → Settings → Developer settings → Personal access tokens.

Pull

Downloading changes from a remote repository AND merging them into the current branch. Equivalent to `git fetch` followed by `git merge`. Done with `git pull`.

Pull request (PR)

A GitHub feature proposing to merge one branch into another. Creates a space for code review, discussion, automated checks and approval before any merge happens.

Push

Uploading local commits to a remote repository. Done with `git push`. Requires authentication for private repos or repos you own.

R

Rebase

Re-applying commits on top of another branch, creating a cleaner linear history. Rewrites commit history. Never rebase commits already pushed to shared branches.

Reflog

A local log of every movement of HEAD — commits, resets, merges, branch switches. Kept for 90 days. The ultimate safety net for recovering apparently lost work.

Remote

A version of a repository hosted on another server — typically GitHub. You push to and pull from remotes. The primary remote is conventionally named `origin`.

Repository (repo)

A folder tracked by Git. Contains all project files plus a hidden `.git` folder storing the entire history.

Revert

Creating a new commit that undoes the changes of a specific previous commit. Safe for shared branches — does not rewrite history. Done with `git revert SHA`.

Runner

In GitHub Actions, the virtual machine that executes workflow jobs. GitHub provides Ubuntu, Windows and macOS runners. Also supports self-hosted runners.

S

Secret

An encrypted value stored securely in GitHub settings. Used in workflow files as `${{ secrets.NAME }}`. Never visible in logs. Used for passwords, tokens and API keys.

SHA hash

The unique 40-character identifier of every commit. Generated by a cryptographic hash of the commit contents. Usually shown as the first 7 characters, e.g. `a3f9c2b`.

Squash

Combining multiple commits into one. Used in interactive rebase to clean up messy history before merging a PR. Also a merge strategy on GitHub — "Squash and merge".

SSH key

A cryptographic key pair used for authenticating with GitHub without typing a password. The private key stays on your machine. The public key is added to your GitHub account.

Staging area

The preparation zone between your working tree and the repository. Changes are moved here with `git add` before being permanently saved with `git commit`. Also called the index.

Stash

Temporarily shelving uncommitted changes to get a clean working tree. Done with `git stash`. Restored with `git stash pop`. Useful when switching branches mid-work.

Step

In GitHub Actions, a single task within a job. Either runs a shell command (`run`) or a marketplace action (`uses`). Steps run sequentially within a job.

Submodule

A Git repository embedded inside another Git repository. Used to include external dependencies at a specific commit. Managed with `git submodule` commands.

T

Tag

A named reference to a specific commit. Unlike branches, tags do not move when new commits are made. Used to mark release versions like `v1.0.0`. Created with `git tag`.

Tracking branch

A local branch set up to follow a remote branch. Git knows which remote branch to push to and pull from automatically. Set up with `git push -u origin main`.

Trigger

In GitHub Actions, the event that starts a workflow — such as a push, pull request, schedule or manual dispatch. Defined in the `on:` section of the workflow file.

Trunk-Based Development

A branching strategy where everyone commits to the main branch (trunk) directly or through very short-lived branches. Requires strong automated testing and feature flags.

U

Untracked file

A file in the working tree that Git has never seen before — it has not been added or committed. Shown in `git status` under "Untracked files". Add with `git add filename`.

Upstream

The original repository that a fork was created from. Also used to describe the remote branch that a local branch tracks. Add with `git remote add upstream URL`.

V

Version control

A system that records changes to files over time so you can recall specific versions later. Git is the world's most popular version control system.

W

Working tree

The actual files on your disk as they currently exist — what you see in your file explorer and edit in your editor. Changes here are not saved to Git history until staged and committed.

Workflow

In GitHub Actions, an automated process defined in a YAML file in `.github/workflows/`. Triggered by Git events. Contains one or more jobs which contain steps.

Y

YAML

Yet Another Markup Language. The file format used for GitHub Actions workflow files. Uses indentation to define structure. Files end in `.yml` or `.yaml`.

Quick Term Lookup

Term	Where explained
Alias	Chapter 3, Annex B
Bisect	Chapter 8
Blame	Chapter 8
Branch	Chapter 5
Cherry-pick	Chapter 8
CI/CD	Chapter 9
Clone	Chapter 6
Commit	Chapter 4
Detached HEAD	Chapter 5
Fetch	Chapter 6
Fork	Chapter 7
GitHub Actions	Chapter 9
GitHub Flow	Chapter 7
GitHub Pages	Chapter 10
.gitignore	Chapter 4
HEAD	Chapter 2
Interactive rebase	Chapter 8
Merge	Chapter 5
Merge conflict	Chapter 5
Origin	Chapter 6
Pull	Chapter 6
Pull request	Chapter 7
Push	Chapter 6

Rebase	Chapter 5
Reflog	Chapter 8
Remote	Chapter 6
Repository	Chapter 4
Revert	Chapter 8
Reset	Chapter 8
SHA hash	Chapter 2
Squash	Chapter 7, Chapter 8
Staging area	Chapter 4
Stash	Chapter 8
Tag	Chapter 2
Tracking branch	Chapter 6
Working tree	Chapter 4

End of Annex D

If you encounter a term not listed here, it may be specific to a particular tool or framework built on top of Git. The official Git documentation at https://git-scm.com/doc is the authoritative reference for all Git terminology.